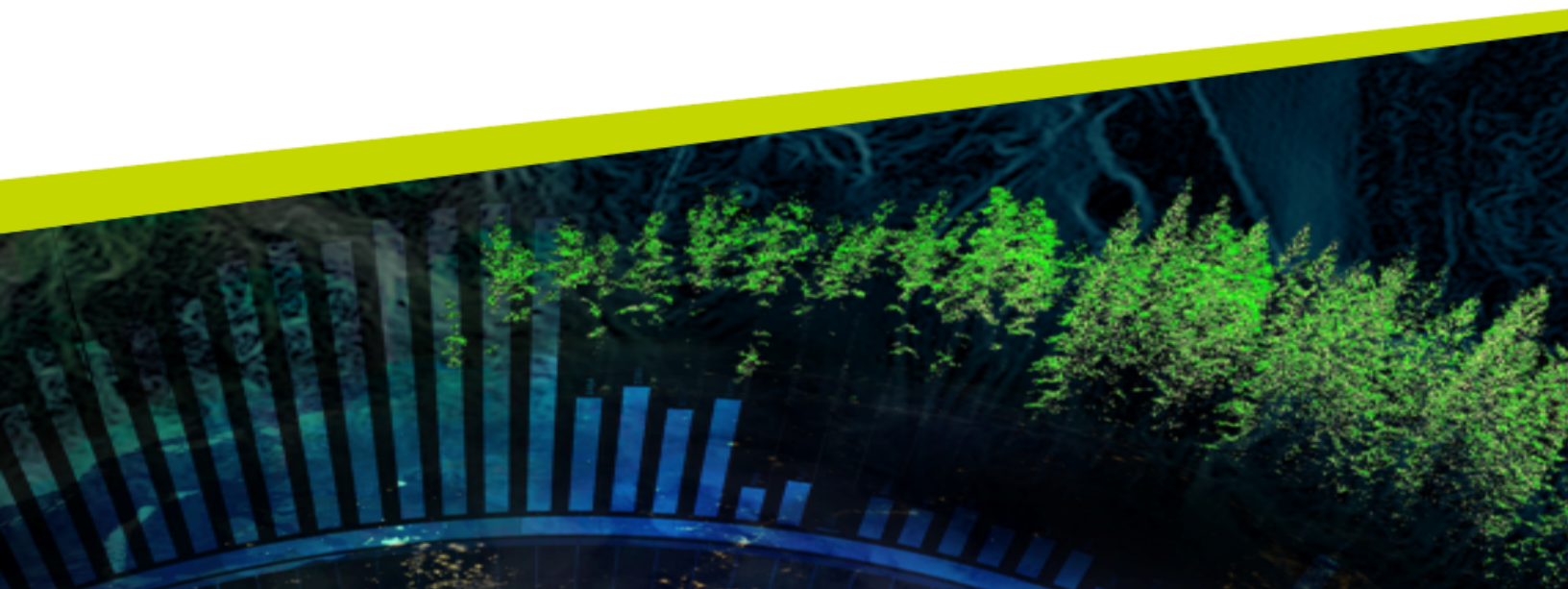




INFERENCIA Y MODELOS ESTADÍSTICOS

José Luis Jara V. y Jacqueline Köhler C.



CAPÍTULO 14. REGRESIÓN LINEAL MÚLTIPLE

En el capítulo anterior conocimos los principios detrás de la regresión lineal, considerando para ello una única variable predictora y una variable de respuesta. Sin embargo, en la vida real es más frecuente que un fenómeno pueda ser explicado por **varias variables**. En consecuencia, en este capítulo presentaremos un nuevo modelo lineal más complejo: la regresión lineal múltiple (RLM), correspondiente al caso de una única respuesta con múltiples predictores. Para ello tomaremos como base los textos de Field et al. (2012, pp. 245-311), Diez et al. (2017, pp. 372-385) y Fox y Weisberg (2018).

14.1 MODELO DE RLM

Una regresión lineal con múltiples variables tiene la forma que se presenta en la ecuación 14.1:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p \quad (14.1)$$

donde:

- Cada x_i es un predictor.
- Cada β_i corresponde a un parámetro del modelo.
- p es la cantidad de predictores.
- $\hat{y}$ es una estimación de la respuesta.

Una vez más, podemos estimar los p parámetros mediante el **método de mínimos cuadrados** que busca minimizar la suma de los cuadrados de los residuos (ecuación 14.2). Este proceso que se vuelve más complejo a medida que aumenta la cantidad de predictores por lo que suele hacerse mediante el uso de software.

$$\min \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (14.2)$$

Para los ejemplos de este capítulo usaremos una vez más el conjunto de datos `mtcars`, considerando vehículos entre 2 y 5 mil libras de peso, cuyas variables describimos en la tabla 13.1. Como punto de partida, recordemos la regresión lineal simple (RLS) para predecir la potencia del motor a partir del volumen útil de sus cilindros (figura 13.7).

Ahora consideremos su extensión a una RLM agregando como un segundo predictor el peso del vehículo (columna `wt`), modelo que podemos obtener mediante el script 14.1.

Script 14.1: regresión lineal para predecir la potencia del motor (en vehículos entre 2 y 5 mil libras) a partir de dos predictores: el volumen útil de sus cilindros y el peso del vehículo.

```
1 library(dplyr)
2 library(scatterplot3d)
3
4 # Cargar y filtrar los datos
5 datos <- mtcars |> filter(wt > 2 & wt < 5)
6
7 # Ajustar y mostrar un modelo de LRM para la potencia del motor
8 modelo <- lm(hp ~ disp + wt, data = datos)
9 print(summary(modelo))
```

Vemos que el procedimiento para construir un modelo de RLM en R es el mismo que usamos en el capítulo anterior, pero ahora para ajustar el modelo tenemos que combinar ambos predictores en el lado derecho de la fórmula (línea 8 del script). Al desplegar el resumen del modelo en pantalla (línea 9), vemos que la salida, mostrada en la figura 14.1, también es muy similar a lo visto para RLS, solo que ahora se reporta una línea extra bajo el encabezado `Coefficients` con el detalle para el nuevo predictor `wt`.

```

Call:
lm(formula = hp ~ disp + wt, data = datos)

Residuals:
    Min       1Q   Median       3Q      Max
-62.701 -38.975   0.804   7.635 147.997

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  41.1736    66.4604   0.620  0.54194
disp         0.5205     0.1556   3.345  0.00293 **
wt          -3.0407     28.0046  -0.109  0.91452
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 46.63 on 22 degrees of freedom
Multiple R-squared:  0.5537, Adjusted R-squared:  0.5131
F-statistic: 13.65 on 2 and 22 DF, p-value: 0.00014

```

Figura 14.1: modelo de RLM con dos predictores obtenido con el script 14.1.

Como en este caso tenemos dos predictores, lo que se ajusta ya no es una recta, sino **un plano**. De acuerdo a los coeficientes reportados en la figura 14.1, este plano corresponde a:

$$\begin{aligned}\widehat{hp} &= b_0 + b_1 \text{ disp} + b_2 \text{ wt} \\ &\approx 41,174 + 0,521 \cdot \text{disp} - 3,041 \cdot \text{wt}\end{aligned}\quad (14.3)$$

De este modo, b_0 (la intercepción) corresponde a la posición del plano con respecto al origen, b_1 es el parámetro ajustado para el volumen útil de los cilindros del motor y b_2 es el parámetro ajustado para el peso del vehículo. El script 14.2 permite obtener una representación gráfica de este modelo, la que es presentada en la figura 14.2.

Script 14.2: (continuación del script 14.1) gráfico 3D del plano de regresión obtenido para el ejemplo.

```

11 # Graficar el modelo ajustado, diferenciando valores sobre y bajo el plano
12 i_color <- 1 + (resid(modelo) > 0)
13 g <- scatterplot3d(
14   datos[["disp"]], datos[["wt"]], datos[["hp"]], type = "p", angle = 50,
15   pch = 16, color = c("steelblue1", "steelblue4")[i_color],
16   xlab = bquote("Volumen útil de los cilindros" ~ group("[", "in"^3, "]")),
17   ylab = "Peso [lb x 1000]\n\n",
18   zlab = "Potencia [hp]",
19   mar = c(3, 3, 1, 0) + 0.1
20 )
21 g$plane3d(modelo, draw_lines = TRUE, lty = "dotted")

```

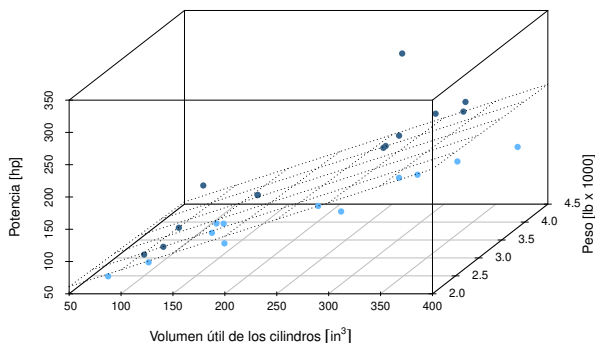


Figura 14.2: plano ajustado para la RLM ejemplo con dos predictores numéricos.

Un aspecto importante del método de mínimos cuadrados con múltiples variables es que busca determinar el **efecto parcial** que cada predictor ejerce en la respuesta controlando por la influencia de los otros predictores, por lo que cada coeficiente β_i de un modelo de RLM, asociado al i -ésimo predictor, representa el cambio esperado que se produce en la respuesta al incrementar dicho predictor en **una unidad**, manteniendo **constantes** todos los demás predictores.

Para el ejemplo, en la figura 14.1 vemos que si se aumenta el volumen disponible de los cilindros de un vehículo en una pulgada cúbica, la potencia del motor aumentaría en un poco más de medio caballo de fuerza ($b_1 \approx 0,521$), mientras que si se aumenta su peso en mil libras, su potencia disminuiría en aproximadamente tres caballos de fuerza ($b_2 \approx -3,041$).

Así, ya no tiene sentido hablar de la *pendiente de la recta* al momento de interpretar los parámetros del modelo. Para este caso, con dos predictores, los valores que están sobre el **plano de regresión** generan residuos positivos, mientras aquellos que están por debajo de este plano generan residuos negativos. En la figura 14.2, los primeros están pintados con un tono azul más oscuro y los segundos en un tono más claro. Si bien esta interpretación gráfica del signo de un residuo sigue siendo válida cuando se tienen más de dos predictores, donde hablaríamos de un **hiperplano de regresión**, es bastante más difícil de dibujar!

Al igual que con la RLS, el hiperplano de regresión ajustado puede ser utilizado para estimar observaciones de la variable salida conociendo el valor de los predictores incluidos en el modelo, siempre y cuando estos valores estén en los rangos considerados durante su ajuste. Para el ejemplo, con el plano de la ecuación 14.3 podemos estimar la potencia de un automóvil construido en EE.UU. a principio los años 70 conociendo el volumen útil de los cilindros de su motor y su peso, cuando estos están, respectivamente, en los rangos 78,7–400,0 pulgadas cúbicas y 2.140–4.070 libras.

El script 14.3 muestra cómo utilizar el modelo ajustado en el script 14.1 para predecir valores de la variable respuesta (**hp**) usando, una vez más, la función `predict()`, a partir de un nuevo conjunto de datos que contiene valores para los predictores incluidos en el modelo (**disp** y **wt**).

Script 14.3: (continuación del script 14.2) uso del modelo de RLM para predecir la potencias de otros vehículos.

```

23 # Definir valores de los predictores para vehículos no incluidos
24 # en el conjunto mtcars.
25 modelos <- c("Chevrolet Vega", "Ford Pinto", "AMC Pacer",
26             "Plymouth Valiant Duster", "Chevrolet Impala Caprice")
27 disp <- c(140, 98, 232, 360, 350)
28 wt <- c(2.2, 2.0, 3.0, 3.5, 2.5)
29 datos_nuevos <- data.frame(disp, wt)
30 rownames(datos_nuevos) <- modelos
31
32 # Usar el modelo para predecir el rendimiento de otros modelos
33 hp_est <- predict(modelo, newdata = datos_nuevos)
34 datos_nuevos <- cbind(datos_nuevos, hp_est)
35
36 # Mostrar los resultados
37 cat("Predicciones:\n")
38 print(datos_nuevos)

```

El resultado de ejecutar el script 14.3 es mostrado en la figura 14.3.

Predicciones:	disp	wt	hp_est
Chevrolet Vega	140	2.2	107.36080
Ford Pinto	98	2.0	86.10593
AMC Pacer	232	3.0	152.81864
Plymouth Valiant Duster	360	3.5	217.92840
Chevrolet Impala Caprice	350	2.5	215.76361

Figura 14.3: predicciones de la potencia de nuevos automóviles con el modelo de RLM del script 14.1.

14.2 PREDICTORES CATEGÓRICOS NO DICOTÓMICOS

Al igual que con RLS, un modelo de RLM puede usar una variable categórica con dos niveles como predictor, haciendo la correspondiente transformación a una variable indicadora. Ahora vamos a extender esta idea para el caso de variables categóricas con d niveles, que debemos convertir en variables indicadores aplicando el siguiente algoritmo:

1. Crear $d - 1$ nuevas variables artificiales.
2. Para cada una de estas nuevas variables:
 - a) Escoger un nivel diferente de la variable original.
 - b) Asignar un 1 a todas las observaciones que tengan ese nivel y un 0 a las restantes.

Para entender mejor esta idea, supongamos que un conjunto de datos tiene la variable categórica `tipo`, con cuatro niveles: **A**, **B**, **C** y **D**. Creamos tres nuevas variables, por ejemplo, `tipo_B`, `tipo_C` y `tipo_D`. La variable `tipo_B` contiene tantas observaciones como la variable original, con un 1 para aquellas observaciones en que `tipo` toma el valor **B** y un 0 para las restantes. Para las variables `tipo_C` y `tipo_D` se procede de manera análoga. Puesto que solo se crean $d - 1$ variables artificiales, se descarta aquella correspondiente al nivel **A** de la variable `tipo`, cuyas observaciones se identifican porque todas las variables indicadoras tienen valor cero.

En R, podemos hacer esta tarea usando la función `dummy(x)` del paquete `dummy`, donde $\mathbf{x}$ es la matriz de datos (`data.frame`) original. La función devuelve una nueva matriz de datos con las variables indicadoras (*dummy*, en inglés) de todos los factores y vectores de strings en $\mathbf{x}$. Existen otros argumentos que permiten obtener variables indicadoras solo para algunos valores categóricos, para obtener variables indicadoras enteras (por omisión son strings), o considerando las categorías presentes en otro conjunto de datos. Esto último es útil cuando pueden existir diferencias en categorías encontradas en los datos que se están siendo procesados y los datos usados para construir el modelo (usualmente llamados **datos de entrenamiento**, como vimos en el capítulo 13).

Por ejemplo, consideremos la tabla 14.4. A la izquierda se muestra una matriz de datos con una variable entera, dos variables categóricas, la primera dicotómica, y la segunda es la variable `tipo` descrita más arriba, y una columna numérica de tipo flotante. A la derecha se presenta la matriz de datos que resulta de aplicar la función `dummy()` a la primera. Podemos apreciar que aparecen variables indicadoras para cada uno de los niveles de las dos variables categóricas.

Matriz de datos original				Resultado de aplicar la función <code>dummy()</code> a la matriz original					
<code>persona</code>	<code>sexo</code>	<code>tipo</code>	<code>valor</code>	<code>sexo_F</code>	<code>sexo_M</code>	<code>tipo_A</code>	<code>tipo_B</code>	<code>tipo_C</code>	<code>tipo_D</code>
1	F	B	1.68	1	0	0	1	0	0
2	F	D	2.79	1	0	0	0	0	1
3	M	A	1.92	0	1	1	0	0	0
4	M	B	2.26	0	1	0	1	0	0
5	M	A	2.10	0	1	1	0	0	0
6	M	C	2.63	0	1	0	0	1	0
7	F	D	2.19	1	0	0	0	0	1
8	M	D	3.62	0	1	0	0	0	1
9	F	D	2.76	1	0	0	0	0	1

Figura 14.4: creación de variables artificiales para una matriz de datos con variables categóricas.

El script 14.4 muestra el funcionamiento de `dummy()` para el ejemplo de la tabla 14.4 (línea 11). Como esta función crea una variable artificial por cada nivel de una variable categórica, debemos descartar una de ellas antes de utilizarlas para construir un modelo de RLM. Así, las líneas 12 y 13 eliminan dos de las variables indicadoras creadas, `sexo_F` y `tipo_A`, respectivamente, que son redundantes. Luego, la línea 14 agrega la variable `valor` a la matriz resultante. Con ello, es posible construir el modelo de RLM (línea 19) y mostrar los coeficientes obtenidos en pantalla (línea 20). El resultado puede verse en la figura 14.5

Script 14.4: creación de variables artificiales para variables categóricas.

```

1 library(dummy)
2
3 # Crear la matriz de datos del ejemplo
4 persona <- 1:9
5 sexo <- c("F", "F", "M", "M", "M", "M", "F", "M", "F")
6 tipo <- c("B", "D", "A", "B", "A", "C", "D", "D", "D")
7 valor <- c(1.68, 2.79, 1.92, 2.26, 2.1, 2.63, 2.19, 3.62, 2.76)
8 datos <- data.frame(persona, sexo, tipo, valor)
9
10 # Crear las variables indicadoras
11 datos_indicadoras <- dummy(datos)
12 datos_indicadoras[["sexo_F"]] <- NULL
13 datos_indicadoras[["tipo_A"]] <- NULL
14 datos_indicadoras[["valor"]] <- datos[["valor"]]
15
16 # Crear y mostrar el modelo de RLM usando variables indicadoras
17 cat("Modelo de RLM con variables indicadoras explícitas\n")
18 cat("-----")
19 modelo <- lm(valor ~ sexo_M + tipo_B + tipo_C + tipo_D, datos_indicadoras)
20 print(modelo)
21
22 # Crear y mostrar el modelo de RLM dejando el trabajo a la función lm()
23 cat("Modelo de RLM con variables indicadoras implícitas\n")
24 cat("-----")
25 modelo_directo <- lm(valor ~ sexo + tipo, datos)
26 print(modelo_directo)

```

```

Modelo de RLM con variables indicadoras explícitas
-----
Call:
lm(formula = valor ~ sexo_M + tipo_B + tipo_C + tipo_D, data = datos.dummy)

Coefficients:
(Intercept)      sexo_M1      tipo_B1      tipo_C1      tipo_D1
      1.154         0.856         0.388         0.620         1.472

Modelo de RLM con variables indicadoras implícitas
-----
Call:
lm(formula = valor ~ sexo + tipo, data = datos)

Coefficients:
(Intercept)      sexoM      tipoB      tipoC      tipoD
      1.154         0.856         0.388         0.620         1.472

```

Figura 14.5: resultado del script 14.4 que construye el mismo modelo de RLM con variables indicadoras explícitas y de forma implícita.

Como adelantamos al estudiar la RLS, la función `lm()` realiza internamente este proceso cuando recibe una variable categórica entre los predictores. Es más, las variables indicadoras eliminadas en el script 14.4 replican el criterio que utiliza `lm()`, que descartada el primer nivel. La línea 25 ajusta el modelo de RLM usando el conjunto de datos original, dejando que la función `lm()` construya las variables indicadoras pertinentes. La línea 26 muestra el modelo que se obtiene con esta alternativa. Si nos fijamos en la figura 14.5, podemos ver que obtenemos el mismo modelo (con los mismos predictores, con iguales coeficientes).

14.3 BONDAD DE AJUSTE DE UN MODELO DE RLM

En el capítulo anterior introdujimos el coeficiente de determinación R^2 como instrumento para evaluar la bondad de ajuste de una regresión lineal simple, puesto que mide la proporción de la varianza total de la variable de salida que es explicada por el modelo. Sin embargo, cuando el modelo es multivariado, esta medida presenta un problema: R^2 aumenta **siempre** que se agregan más predictores al modelo, incluso si

esos predictores no contribuyen significativamente a explicar la variabilidad de los datos, por lo que puede llevar a una idea equivocada de la calidad del modelo.

Así, para evaluar un modelo de RLM tenemos que usar un **coeficiente de determinación ajustado** ($R^2_{\text{adj.}}$), que tenga en cuenta el número de predictores en el modelo. Diferentes autores han propuesto distintas maneras de efectuar este ajuste, una de las cuales presentamos en la ecuación 14.4:

$$R^2_{\text{adj.}} = 1 - \frac{(1 - R^2)(n - 1)}{n - p} \quad (14.4)$$

donde n y p siguen siendo, respectivamente, el número total de observaciones y el número de parámetros del modelo.

Si bien podemos usar este ajuste con un único predictor, la diferencia suele ser irrelevante. En RLM, en cambio, penalizar la **complejidad del modelo** (número de predictores) proporciona una métrica más realista. Un $R^2_{\text{adj.}} < R^2$ indica la presencia de predictores **irrelevantes** que no aportan al ajuste del modelo, mientras que un $R^2_{\text{adj.}} \approx R^2$ sugiere que las variables agregadas sí son relevantes.

Preferir modelos más simples (con menos predictores) es conocido como el **principio de parsimonia**, también llamado “navaja de Occam”, y es un **concepto fundamental** en ciencia, estadística y **aprendizaje automático** (*machine learning*). Este sostiene que ante múltiples explicaciones o modelos candidatos para describir un fenómeno, se debe preferir el **más simple** que sea **consistente con los datos** disponibles, evitando el **exceso de supuestos** o componentes. Versiones del principio se pueden rastrear hasta Aristóteles (Franklin, 2001), y siglos de uso han mostrado una y otra vez que la explicación más sencilla suele ser la mejor, puesto que un modelo simple tiene menos probabilidad de ajustar **ruido** o **variabilidad aleatoria** en los datos, son más fáciles de entender, comunicar y justificar, y es más probable que un modelo parsimonioso generalice mejor a datos no vistos.

Otras alternativas para evaluar la bondad de ajuste bajo este principio son el **criterio de información de Akaike** (AIC) y el **criterio bayesiano de Schwarz** (BIC), que penalizan variables adicionales (a menor valor, mejor modelo). Si bien el cálculo de estas medidas no se detalla aquí por ser un tópico más avanzado, en R pueden obtenerse mediante las funciones `AIC(object)` y `BIC(object)` del paquete básico `stats`, donde `object` es un modelo lineal ya construido, como se ejemplifica en el script 14.5.

Script 14.5: comparación de los dos modelos lineales del ejemplo.

```
1 library(dplyr)
2
3 # Cargar y filtrar los datos
4 datos <- mtcars |> filter(wt > 2 & wt < 5)
5
6 # Ajustar el modelo nulo, sin predictores# solo intercepto
7 modelo_0 <- lm(hp ~ 1, data = datos)
8
9 # Ajustar un modelo con volumen de los cilindros como predictor
10 modelo_1 <- lm(hp ~ disp, data = datos)
11
12 # Ajustar un modelo añadiendo el peso como predictor
13 modelo_2 <- lm(hp ~ disp + wt, data = datos)
14
15 # Mostrar AIC y BIC de los modelos
16 cat("Modelo 0: AIC =", AIC(modelo_0), "\n")
17 cat("Modelo 1: AIC =", AIC(modelo_1), "\n")
18 cat("Modelo 2: AIC =", AIC(modelo_2), "\n\n")
19 cat("Modelo 0: BIC =", BIC(modelo_0), "\n")
20 cat("Modelo 1: BIC =", BIC(modelo_1), "\n")
21 cat("Modelo 2: BIC =", BIC(modelo_2), "\n\n")
22
23 # Comparar los modelos
24 comparacion <- anova(modelo_0, modelo_1, modelo_2)
25 cat("Prueba de bondad de ajuste:\n")
26 cat("-----\n")
27 print(comparacion)
```


El script 14.5 construye tres modelos: el modelo nulo (línea 7), que solo considera el intercepto (por lo que corresponde a la recta horizontal que pasa por la media de la variable de salida), un modelo de RLS (línea 10) que usa el volumen útil de los cilindros del motor como predictor, y un modelo de RLM (línea 13) que extiende este último añadiendo el peso del vehículo (`wt`) como nuevo predictor.

La salida producida por el script puede verse en la figura 14.6, donde apreciamos el modelo nulo registra $AIC \approx 284,0$ y $BIC \approx 286,5$; valores que bajan para el modelo de RLS que exhibe $AIC \approx 265,9$ y $BIC \approx 269,5$; mientras que el modelo de RLM registra valores similares: $AIC \approx 267,9$ y $BIC \approx 272,8$. En consecuencia, bajo estos criterios, pareciera que se obtiene un beneficio al considerar el volumen disponible de los cilindros para predecir la potencia del motor respecto de usar simplemente la media, pero agregar el peso del automóvil como predictor no pareciera aportar a un mejor ajuste a los datos.

Esto no debería sorprendernos, ya que otra forma de conocer cuáles predictores aportan significativamente al ajuste es observar el estadístico t y los valores p asociados a cada predictor de un modelo de RLM. Para el ejemplo, estas se aprecian en las columnas `Pr(t value)` y `Pr(>|t|)` de la figura 14.1, las que indicaban que, a un nivel α usual, no había evidencia de que la variable `wt` hiciera un aporte significativo ($t(22) = -0,109$; $p = 0,914$), mientras que para la variable `disp` sí existía ($t(22) = 3,345$; $p = 0,003$).

Si bien las métricas AIC y BIC nos pueden resultar útiles para comparar dos modelos de regresión lineal, considerando la regla general que un modelo es mejor mientras menor sea su valor, es importante notar que, al ser **medidas relativas**, hasta ahora no contamos con una prueba estadística que nos permita determinar si la diferencia es significativa, por lo que no tenemos cómo determinar a ciencia cierta si dos modelos dados son similares o realmente uno de ellos es de mejor calidad.

Sin embargo, cuando los modelos son **jerárquicos**, es decir, el segundo incorpora nuevos predictores además de mantener los del primer modelo, podemos hacer una prueba de hipótesis usando los coeficientes de determinación. El estadístico de prueba sigue una distribución F con $p_2 - p_1$ y $n - p_2$ grados de libertad, y se calcula mediante la ecuación 14.5:

$$F_{\text{cambio}} = \frac{(n - p_2) (R_2^2 - R_1^2)}{(p_2 - p_1) (1 - R_2^2)} \quad (14.5)$$

donde:

- n : cantidad de observaciones.
- p_1, p_2 : cantidad de parámetros en el primer y segundo modelo, respectivamente, suponiendo $p_1 < p_2$.
- R_1^2, R_2^2 : coeficientes de determinación del primer y segundo modelo, respectivamente.

```
Modelo 0: AIC = 284.0302
Modelo 1: AIC = 265.8756
Modelo 2: AIC = 267.8622

Modelo 0: BIC = 286.468
Modelo 1: BIC = 269.5322
Modelo 2: BIC = 272.7377

Prueba de bondad de ajuste:
Analysis of Variance Table

Model 1: hp ~ 1
Model 2: hp ~ disp
Model 3: hp ~ disp + wt
  Res.Df    RSS Df Sum of Sq      F      Pr(>F)
1      24 107174
2      23  47859   1    59314 27.2802 3.075e-05 ***
3      22  47834   1      26  0.0118  0.9145
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Figura 14.6: resultado del script 14.5 que compara el modelo nulo y los modelos de RLS y RLM ajustados como ejemplos.

Así, para los modelos ejemplos:

$$F_{\text{nulo} \rightarrow \text{RLS}} = \frac{(25-2) \cdot (0,5534-0)}{(2-1) \cdot (1-0,5534)} \approx 28,505 \quad F_{\text{RLS} \rightarrow \text{RLM}} = \frac{(25-3) \cdot (0,5537-0,5534)}{(3-2) \cdot (1-0,5537)} \approx 0,015$$

Como siempre, podemos determinar valores p para estos estadísticos de prueba mediante las llamadas `pf(28.505, 1, 23, lower.tail = FALSE)` y `pf(0.015, 1, 22, lower.tail = FALSE)`, obteniendo como resultado, respectivamente, $p \approx 0,00002$ y $p \approx 0,904$. Esto confirma nuestra intuición inicial: hay evidencia fuerte para inclinarse por que el modelo de RLS se ajusta significativamente mejor a los datos que el modelo nulo, pero no hay evidencia suficiente de que el modelo de RLM construido se ajuste mejor a los datos que el modelo de LRS (suponiendo un nivel de confianza usual como 95 % o 99 %).

Como ya es habitual, R puede hacer estos cálculos por nosotros. La línea 24 del script 14.5 muestra cómo comparar el modelo nulo con el modelo de RLS, y luego este último con el modelo de RLM construidos previamente, todo con solo una llamada a la función `anova(object, ...)`, que recibe como argumentos los **modelos jerárquicos** a comparar y entrega la evaluación del cambio en varianza (F_{cambio}) entre cada par de modelos. En cada una de estas comparaciones, si el estadístico lleva a un valor p significativo, podemos concluir que el modelo más complejo (con más predictores) se ajusta mejor a los datos que el más simple, es decir, produce **residuos significativamente menores** en promedio.

El resultado para el ejemplo se presenta en la figura 14.6 bajo el título “Prueba de bondad de ajuste”, donde podemos ver que el modelo de RLS consigue reducir significativamente la variabilidad no explicada con respecto al modelo nulo (de 107.174 a 47.859; $F(1, 23) = 27,280$; $p < 0,001$), mientras que la reducción adicional que consigue el modelo de RLM es insignificante (de 47.859 a 47.834; $F(1, 22) = 0,012$; $p = 0,915$). Este resultado es similar al que obtuvimos anteriormente, con diferencias que se deben al redondeo.

14.4 SELECCIÓN DE PREDICTORES

Considerando el principio de parsimonia, es claro que debemos contar con alguna estrategia que nos permita determinar qué predictores incluir en un modelo de RLM, de manera que sean relevantes para mejorar su ajuste a los datos pero que, al mismo tiempo, no incluya más predictores de los necesarios para mantener su generalidad. Existen diversas alternativas para esta tarea que exploraremos a continuación.

14.4.1 Regresión jerárquica

El método más adecuado, aunque también el más complejo, es la **regresión jerárquica**. Es el que debemos considerar al momento de intentar probar una teoría y consiste en comenzar por incorporar en primer lugar aquellos predictores ya conocidos, en orden de importancia, en base a investigaciones previas (requiere una revisión bibliográfica). Una vez incorporados todos los predictores ya conocidos, podemos incorporar otros nuevos si creemos que existen **buenas y justificadas razones** para ello. Antes de la masificación de los computadores y de entornos como R, ésta era la única alternativa viable!

En R, podemos realizar este método con ayuda de la función `update(object, formula)`, que nos permite incorporar o quitar variables del modelo, donde:

- **object**: modelo previamente ajustado, en este caso con `lm()`.
- **formula**: actualización de la fórmula para el nuevo modelo. La especificación “`. ~ .`” indica “todo lo que hay al lado izquierdo de la fórmula” (el primer punto) y “todo lo que hay al lado derecho de la fórmula” (el segundo punto).

En el caso del ejemplo, existe una teoría previa basada en la física, pero que es difícil de transferir a modelos de vehículos específicos. Podemos revisar esta teoría y complementarla con múltiples recursos documentales que discuten sobre la potencia y rendimiento de motores de automóviles, algunos de ellos se refieren a los de la época de interés, como por ejemplo:

Aparicio Izquierdo, F., Vera Álvarez, C., & Díaz López, V. (2001). Universidad Politécnica de Madrid. Caliber. (s.f.). Car Design Through Time: From Classic to Contemporary. Caliber.com.

Motor Trend Publications. Motor Trend Magazine collection. From the Collections of The Henry Ford. <https://www.thehenryford.org/collections-and-research/digital-collections>.

Seger, E.E., & Brink, R.S. (1972). Trends of Vehicle Dimensions and Performance Characteristics from 1960 through 1970. Research Record, 1.

Revisando estos textos, encontramos que en los automóviles de los años 70, la potencia del motor estaba influenciada por varias variables mecánicas, de diseño y ambientales. A continuación, se describen las principales variables, más o menos en orden de importancia:

1. El volumen total de los cilindros del motor, pues motores con mayor volumen tienden a generar más potencia debido a su capacidad para quemar más mezcla de combustible y aire en cada ciclo.
2. Número de cilindros en el motor, por la misma razón anterior.
3. Relación de compresión entre el volumen del cilindro con el pistón en su posición más baja frente a su posición más alta, pues a mayor volumen más energía se extrae de la combustión.
4. Cantidad y tipo de carburadores, que influye en la capacidad del motor para mezclar aire y combustible de manera eficiente.
5. Eficiencia del sistema de escape, ya que una liberación rápida de gases mejora el rendimiento.
6. Peso del vehículo, ya que un auto pesado requiere más potencia, por lo que los fabricantes usan motores más potentes para autos grandes.
7. Configuración geométrica del motor (V, en línea, etc.), ya que influye en cómo se distribuye la potencia.
8. Ajuste de engranajes, pues relaciones de transmisión más ajustadas aumenta la entrega de potencia.
9. Leyes ambientales, porque en los años 70, especialmente en EE. UU., surgieron nuevas regulaciones que limitaron el uso de tecnologías que maximizaban la potencia en favor de reducir emisiones contaminantes.

Algunas de estas características no están incluidas en el conjunto de datos `mtcars`, otras están algo relacionadas a las variables registradas. Haciendo el análisis, podríamos seleccionar las siguientes variables, en orden de importancia: `disp`, `cyl`, `carb`, `wt` y `vs`.

El script 14.6 presenta la aplicación de regresión jerárquica basada en esta información, cuyo resultado se presenta en la figura 14.7. Comenzamos filtrando y preparando los datos (líneas 4–5), asegurándonos que las variables que son categóricas tengan tipo `factor`, de manera que la función `lm()` sepa que debe usar variables indicadoras y no tratarlas como valores numéricos.

Luego, en la línea 8, construimos un modelo con un único predictor: el volumen de los cilindros (`disp`), el que, como sabemos, consigue un mejor ajuste que el modelo nulo (figura 14.6). En segundo lugar consideramos agregar al modelo la variable categórica `cyl` (línea 11), con el número de cilindros del vehículo. Como esta tiene directa relación con el predictor anterior, es poco probable que aporte información nueva. Esta intuición es confirmada al comparar ambos modelos en la línea 12 del script ($F(2, 21) = 2,536$; $p = 0,103$).

Consideramos entonces utilizar la variable categórica `carb`, con el número de carburadores con que cuenta el automóvil, como segundo predictor, en reemplazo de la variable `cyl` (línea 16). En este caso, la nueva variable sí aporta al modelo, mejorando su ajuste como indica la salida de la comparación en la línea 18 ($F(5, 18) = 14,080$; $p < 0,001$).

Ya tenemos un modelo de RLM con dos predictores. Como los coeficientes han cambiado, reevaluamos que la variable `cyl` sigue siendo irrelevante (línea 22), pero nos llevamos la sorpresa de que ahora esta ¡sí aporta al modelo! ($F(2, 16) = 17,197$; $p < 0,001$). Por esta razón, mantenemos el nuevo predictor y ya contamos con un modelo de RLM con tres predictores.

Siguiendo con los resultados de nuestra investigación bibliográfica (superficial), intentamos agregar el peso del vehículo (`wt`) como un cuarto predictor para el modelo en las líneas 28–30. Pero esta variable resulta irrelevante ($F(1, 15) = 0,395$; $p = 0,539$) para el ajuste, por lo que se descarta.

Script 14.6: aplicación de regresión jerárquica para construir un modelo de RLM para predecir la potencia del motor en automóviles que pesan entre 2 y 5 mil libras construidos en los años 70.

```

1 library(dplyr)
2
3 # Cargar y filtrar los datos
4 datos <- mtcars |> filter(wt > 2 & wt < 5) |>
5   mutate(across(c("cyl", "vs", "am", "gear", "carb"), as.factor))
6
7 # Ajustar el modelo inicial con el volumen de los cilindros como predictor
8 modelo_1 <- lm(hp ~ disp, data = datos)
9
10 # Incorporar al modelo el número de cilindros y verificar su utilidad
11 modelo_2 <- update(modelo_1, . ~ . + cyl)
12 print(anova(modelo_1, modelo_2), signif.legend = FALSE)
13 # Como era esperable, la variable "cyl" no aporta al ajuste del modelo
14
15 # Reemplazar el número de cilindros por el número de carburadores y verificar su utilidad
16 modelo_3 <- update(modelo_2, . ~ . - cyl + carb)
17 cat("\n")
18 print(anova(modelo_1, modelo_3), signif.legend = FALSE)
19 # La variable "carb" sí genera un mejor ajuste, por lo que la mantendremos en el modelo
20
21 # Y en este último modelo, la variable "cyl" sigue siendo irrelevante?
22 modelo_4 <- update(modelo_3, . ~ . + cyl)
23 cat("\n")
24 print(anova(modelo_3, modelo_4), signif.legend = FALSE)
25 # Ahora la variable "cyl" ¡sí ayuda a obtener un mejor modelo!
26
27 # Incorporar al modelo el peso del vehículo y verificar su utilidad
28 modelo_5 <- update(modelo_4, . ~ . + wt)
29 cat("\n")
30 print(anova(modelo_4, modelo_5), signif.legend = FALSE)
31 # Vemos que el peso no aporta a un mejor ajuste. Probablemente está muy relacionado
32 # al número de cilindros y carburadores del motor, ya considerados en el modelo.
33
34 # Reemplazar el peso del vehículo por el tipo de motor y verificar su utilidad
35 modelo_6 <- update(modelo_5, . ~ . - wt + vs)
36 cat("\n")
37 print(anova(modelo_4, modelo_6), signif.legend = FALSE)
38 # Vemos que tipo de motor tampoco ayuda a conseguir un mejor modelo
39
40 # Mostrar el modelo obtenido
41 cat("\n\nModelo obtenido con regresión jerárquica:\n")
42 cat("-----\n")
43 print(summary(modelo_4), signif.legend = FALSE)

```

Solo nos queda probar si el tipo de motor (`vs`) es un predictor valioso. Lo agregamos al modelo en reemplazo del peso del vehículo (línea 35 del script 14.6). Al comparar en la línea 37, encontramos que también debemos descartar esta variable pues no ayuda a mejorar el ajuste ($F(1, 15) = 0,093$; $p = 0,765$).

Así, en definitiva, nos quedamos con el modelo que considera tres predictores: el volumen de los cilindros del motor (`disp`), la cantidad de carburadores con que cuenta el vehículo (`carb`) y el número de cilindros que utiliza (`cyl`).

La línea 43 del script 14.6 despliega los detalles de este modelo en pantalla. En la parte final de la la figura 14.7 (columna derecha), podemos ver que la variable `carb` aporta con cinco variables indicadoras al modelo, mientras que la variable `cyl` aporta con dos. Algo que también podríamos haber notado con las reducciones en los grados de libertad de las pruebas F que el script fue realizando.

El modelo final logra un mucho mejor ajuste que el modelo nulo, disminuyendo la suma de los residuos cuadrados de 107.174 a 3.094, más de un 97 % de reducción.

(Líneas 12, 17–18, 23–24, 29–30)		(Líneas 36–37, 41–43)	
<pre> Analysis of Variance Table Model 1: hp ~ disp Model 2: hp ~ disp + cyl Res.Df RSS Df Sum of Sq F Pr(>F) 1 23 47859 2 21 38549 2 9310.7 2.5361 0.1031 Analysis of Variance Table Model 1: hp ~ disp Model 2: hp ~ disp + carb Res.Df RSS Df Sum of Sq F Pr(>F) 1 23 47859 2 18 9745 5 38114 14.08 1.096e-05 *** Analysis of Variance Table Model 1: hp ~ disp + carb Model 2: hp ~ disp + carb + cyl Res.Df RSS Df Sum of Sq F Pr(>F) 1 18 9745.3 2 16 3094.1 2 6651.2 17.197 0.0001033 *** Analysis of Variance Table Model 1: hp ~ disp + carb + cyl Model 2: hp ~ disp + carb + cyl + wt Res.Df RSS Df Sum of Sq F Pr(>F) 1 16 3094.1 2 15 3014.7 1 79.433 0.3952 0.539 </pre>		<pre> Analysis of Variance Table Model 1: hp ~ disp + carb + cyl Model 2: hp ~ disp + carb + cyl + vs Res.Df RSS Df Sum of Sq F Pr(>F) 1 16 3094.1 2 15 3075.1 1 19.074 0.093 0.7645 Modelo obtenido con regresión jerárquica: ----- Call: lm(formula = hp ~ disp + carb + cyl, data = datos) Residuals: Min 1Q Median 3Q Max -30.422 -3.138 0.000 2.107 28.815 Coefficients: Estimate Std. Error t value Pr(> t) (Intercept) 41.6148 15.7907 2.635 0.017997 * disp 0.4762 0.1144 4.162 0.000734 *** carb2 -19.0440 8.8794 -2.145 0.047671 * carb3 27.9118 16.1811 1.725 0.103794 carb4 57.1928 10.1001 5.663 3.53e-05 *** carb6 120.9125 17.9545 6.734 4.80e-06 *** carb8 170.9128 18.4288 9.274 7.75e-08 *** cyl6 -56.5693 13.9459 -4.056 0.000917 *** cyl8 -20.8492 29.1123 -0.716 0.484212 Residual standard error: 13.91 on 16 degrees of freedom Multiple R-squared: 0.9711, Adjusted R-squared: 0.9567 F-statistic: 67.28 on 8 and 16 DF, p-value: 7.366e-11 </pre>	

Figura 14.7: resultado del script 14.6 con el ejemplo de regresión jerárquica.

14.4.2 Regresión paso a paso

Si en lugar de probar una teoría, lo que queremos es **explorar los datos**, podemos usar otras estrategias para seleccionar el conjunto de predictores para un modelo de LRM:

Selección hacia adelante. A partir de un **modelo nulo**, se escoge la variable **más prometedora** para agregarla al modelo. Hay varias opciones para decidir cuál es esta variable más prometedora, que puede ser aquella que exhiba la correlación más alta con la variable de respuesta, o la que conlleva el mayor aumento del coeficiente de determinación, o la que disminuye más el AIC del modelo, entre otras alternativas. Si el predictor seleccionado incrementa la capacidad predictiva del modelo, se retiene de forma definitiva. En caso contrario, se descarta su inclusión. El procedimiento se repite y en cada iteración se escoge la siguiente variable más prometedora que aún no ha sido evaluada. Si ninguno de los posibles predictores restantes promete mejorar el modelo que se tiene, la búsqueda de nuevos predictores se detiene.

Eliminación hacia atrás. Es el proceso inverso a la selección hacia adelante, puesto que se comienza desde un **modelo completo** que incluye todas las variables disponibles, para luego ir eliminando predictores, uno a uno. Invirtiendo los criterios anteriores, se elige el predictor que **menos aporta** al modelo. Si la eliminación este último mejora el modelo de acuerdo al criterio utilizado, se elimina de forma definitiva y se reevalúa la contribución de los predictores que aún se encuentran en el modelo. Una vez más, el proceso se repite hasta que no es posible eliminar un predictor sin perjudicar su ajuste.

Regresión escalonada. Combina los enfoques anteriores. Comienza desde un modelo vacío o completo y alterna entre agregar y eliminar predictores, evaluando en cada paso si es necesario incluir o excluir una variable según el criterio utilizado, de forma de reducir el efecto debido al orden de incorporación de los predictores.

Como estos métodos seleccionan un predictor en cada iteración, ya sea para agregarlo o para quitarlo del modelo de RLM, se les conocen como métodos de selección de predictores paso a paso o, más simplemente, **regresión paso a paso** (*stepwise regression*, en inglés).

Es crucial reiterar que estas herramientas solo son **éticamente aceptables** en etapas de exploración de datos. Para entender el riesgo de aplicarlas sin un marco conceptual sólido, tomemos como referencia el estudio de Montero Muñoz et al. (2012) sobre el uso de antibióticos en pacientes mayores de 80 años. En su investigación, el equipo seleccionó exclusivamente variables con respaldo clínico. En contraste, un equipo poco ético podría actuar de forma oportunista (e incluso malintencionada) al incorporar variables ajenas al problema, como el monto de la pensión de las personas, solo para ver “qué resulta” o forzar una narrativa conveniente. Aquí, el peligro ya no es solo tropezar con un hallazgo accidental (una simple correlación espuria por azar), sino instrumentalizarlo activamente. Publicar un resultado sesgado que sugiera que un antibiótico es más nocivo para ancianos con bajas pensiones no solo carecería de validez científica, sino que pondría en peligro la vida de personas reales al alterar la práctica médica por negligencia o intereses particulares.

Los métodos de regresión paso a paso están disponibles en R con ayuda de la ya conocida función `update()` y las funciones `add1(object, scope, test)` y `drop1(object, scope, test)`, donde:

- **object**: modelo de regresión base.
- **scope**: fórmula que especifica los potenciales predictores que se podrían agregar (o quitar), o un modelo de regresión creado con esta fórmula. También puede ser la matriz de datos con los potenciales predictores (que no incluya la variable de salida). En el caso de la función `drop1()`, puede omitirse para indicar que todos los predictores del modelo pueden ser candidatos a eliminación.
- **test**: tipo de la prueba de hipótesis a aplicar al comparar el modelo base y el potencial nuevo modelo. Por omisión tiene valor `"none"`, es decir no aplica prueba alguna. Se puede indicar el valor `"F"` para aplicar la prueba basada en la reducción de varianza no explicada que hemos usado con la función `anova()` al comparar modelos de RLM (y que es la que reporta la función `summary()` para un modelo de regresión lineal). Otra alternativa es `"Chisq"` para aplicar la prueba de razón de verosimilitud (LRT, del inglés *likelihood ratio test*) que discutiremos más adelante, en el próximo capítulo.

Las funciones `add1()` y `drop1()` evalúan (individualmente) la incorporación o eliminación, respectivamente, de cada potencial predictor al modelo base y entrega algunas métricas del efecto que su incorporación o eliminación tendría en un nuevo modelo. Para ser más “ordenados”, podemos usar la función `tidy()` del paquete `broom` para convertir la salida de estas funciones en una matriz de datos de clase `"tbl"` (`tibble`) con las siguientes columnas:

- **term** que indica la variable considerada para ser agregada o quitada del modelo base.
- **df** con la variación de los grados de libertad respecto del modelo base.
- **sumsq** que muestra en cuánto el nuevo modelo reduciría o aumentaría, respectivamente, la varianza no explicada respecto del modelo base.
- **rss** con la cantidad de varianza no explicada que quedaría en los residuos del nuevo modelo.
- **AIC** que indica el valor AIC que tendría el nuevo modelo.
- **statistic** que reporta el valor del estadístico F al comparar el nuevo modelo con el base. Obviamente esta columna solo aparece cuando se ha especificado que se aplique una prueba F en cada caso por medio del argumento `test`.
- **p.value** que reporta el valor p de la prueba de igualdad de ajuste, por lo que solo se incluye cuando se ha solicitado una prueba de igualdad de ajuste por medio del argumento `test`.

El script 14.7 muestra cómo se pueden ir usando las funciones `add1()` y `drop1()` para la construcción de un modelo de RLM usando regresión paso a paso. El resultado de ejecutar este script puede verse en la figura 14.8: en el lado izquierdo se aprecia lo obtenido con dos pasos de selección hacia adelante, mientras que en el lado derecho lo que obtenemos con dos pasos de eliminación hacia atrás.

Script 14.7: regresión paso a paso para construir modelos de RLM para el ejemplo con `mtcars`.

```

1 library(broom)
2 library(dplyr)
3 library(formatR)
4 library(scales)
5
6 # Cargar y filtrar los datos
7 datos <- mtcars |> filter(wt > 2 & wt < 5) |>
8   mutate(across(c("cyl", "vs", "am", "gear", "carb"), as.factor))
9
10 # Ajustar el modelo nulo y el modelo completo
11 nulo <- lm(hp ~ 1, data = datos)
12 completo <- lm(hp ~ ., data = datos)
13
14 ##### Selección hacia adelante #####
15
16 # Evaluar las variables para seleccionar el primer predictor
17 paso <- add1(nulo, scope = completo, test = "F") |> tidy() |>
18   mutate(p.value = format(pvalue(p.value), justify = "right")) |> arrange(AIC)
19 cat("", paste("Modelo:", format(formula(nulo))), format(paso), sep = "\n")
20
21 # Agregar la variable que logra la mayor reducción en AIC
22 modelo <- update(nulo, . ~ . + cyl)
23
24 # Evaluar las variables para seleccionar el segundo predictor
25 paso <- add1(modelo, scope = completo, test = "F") |> tidy() |>
26   mutate(p.value = format(pvalue(p.value), justify = "right")) |> arrange(AIC)
27 cat("", paste("Modelo:", format(formula(modelo))), format(paso), sep = "\n")
28
29 # Agregar la variable que logra la mayor reducción en AIC
30 modelo <- update(modelo, . ~ . + carb)
31
32 # Mostrar los coeficientes del modelo conseguido
33 cat("\nCoeficientes para los predictores del modelo obtenido:\n")
34 print(round(modelo[["coefficients"]], 3), width = 56)
35
36 ##### Selección hacia atrás #####
37
38 # Evaluar la eliminación de uno de los predictores del modelo
39 paso <- drop1(completo, test = "F") |> tidy() |>
40   mutate(p.value = format(pvalue(p.value), justify = "right")) |> arrange(desc(statistic))
41 fmla_str <- tidy_source(text = format(formula(completo)), width.cutoff = 47, output = FALSE)
42 cat("", paste("Modelo:", fmla_str[[1]]), format(paso), sep = "\n")
43
44 # Quitar el predictor que menos aporta (con menor estadístico F)
45 modelo <- update(completo, . ~ . - wt)
46
47 # Evaluar la eliminación de uno de los predictores del modelo
48 paso <- drop1(modelo, test = "F") |> tidy() |>
49   mutate(p.value = format(pvalue(p.value), justify = "right")) |> arrange(desc(statistic))
50 fmla_str <- tidy_source(text = format(formula(modelo)), width.cutoff = 47, output = FALSE)
51 cat("", paste("Modelo:", fmla_str[[1]]), format(paso), sep = "\n")
52
53 # Quitar el predictor que menos aporta (con menor estadístico F)
54 modelo <- update(modelo, . ~ . - drat)
55
56 # Mostrar los coeficientes del modelo conseguido
57 cat("\nCoeficientes para los predictores del modelo obtenido:\n")
58 print(round(modelo[["coefficients"]], 3), width = 56)

```

Como en el script anterior, comenzamos filtrando los datos (línea 7) y convirtiendo en factores las variables no numéricas (línea 8). A continuación, en la línea 11 definimos el modelo nulo usando la fórmula `hp ~ 1`, que indica que la estimación de la variable salida `hp` es una constante (su media observada). Similarmente, en la línea 12 se define el modelo completo usando la fórmula `hp ~ .`, que indica que para la estimación de la variable salida se ha de considerar todos los predictores disponibles en la matriz de datos.

Selección hacia adelante

```
Modelo: hp ~ 1
# A tibble: 11 x 7
  term      df  sumsq    rss    AIC statistic p.value
<chr> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
1 cyl      2 68625. 38549. 190. 19.6    "<0.001"
2 qsec     1 61023. 46151. 192. 30.4    "<0.001"
3 mpg      1 60602. 46572. 192. 29.9    "<0.001"
4 disp     1 59314. 47859. 193. 28.5    "<0.001"
5 carb     5 61903. 45271. 200. 5.20    " 0.004"
6 vs       1 44410. 62763. 200. 16.3    "<0.001"
7 gear     2 44037. 63137. 202. 7.67    " 0.003"
8 wt       1 35016. 72158. 203. 11.2    " 0.003"
9 <none>    NA    NA    107174. 211. NA      "  NA"
10 drat     1 5585. 101589. 212. 1.26    " 0.272"
11 am       1 2.40 107171. 213. 0.000516 " 0.982"

Modelo: hp ~ cyl
# A tibble: 10 x 7
  term      df  sumsq    rss    AIC statistic p.value
<chr> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
1 carb     5 32105. 6444. 155. 16.9    "<0.001"
2 gear     2 18503. 20046. 177. 9.23    " 0.001"
3 qsec     1 11620. 26929. 183. 9.06    " 0.007"
4 drat     1 10241. 28308. 184. 7.60    " 0.012"
5 am       1 9686. 28863. 184. 7.05    " 0.015"
6 mpg      1 4222. 34327. 189. 2.58    " 0.123"
7 <none>    NA    NA    38549. 190. NA      "  NA"
8 wt       1 948. 37600. 191. 0.530    " 0.475"
9 vs       1 388. 38161. 191. 0.213    " 0.649"
10 disp     1 0.0387 38549. 192. 0.0000211 " 0.996"

Coeficientes para los predictores del modelo obtenido:
(Intercept)    cyl6      cyl8    carb2
 99.899    -14.246    95.098   -21.572
 carb3      carb4      carb6    carb8
-14.996    41.772    89.348   140.004
```

Selección hacia atrás

```
Modelo: hp ~ mpg + cyl + disp + drat + wt + qsec + vs +
am + gear + carb
# A tibble: 11 x 7
  term      df  sumsq    rss    AIC statistic p.value
<chr> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
1 carb     5 9543. 9911. 174. 41.5    "<0.001"
2 disp     1 1005. 1373. 132. 21.9    " 0.002"
3 cyl      2 1931. 2299. 143. 21.0    "<0.001"
4 am       1 714. 1081. 126. 15.5    " 0.004"
5 vs       1 445. 813. 119. 9.68    " 0.014"
6 gear     2 579. 947. 121. 6.30    " 0.023"
7 qsec     1 253. 621. 112. 5.51    " 0.047"
8 mpg      1 224. 592. 111. 4.86    " 0.058"
9 drat     1 71.5 439. 104. 1.56    " 0.248"
10 wt       1 5.12 373. 99.6 0.111    " 0.747"
11 <none>    NA    NA    368. 101. NA      "  NA"

Modelo: hp ~ mpg + cyl + disp + drat + qsec + vs + am +
gear + carb
# A tibble: 10 x 7
  term      df  sumsq    rss    AIC statistic p.value
<chr> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
1 carb     5 10545. 10918. 174. 50.9    "<0.001"
2 disp     1 1429. 1802. 137. 34.5    "<0.001"
3 cyl      2 2175. 2548. 144. 26.2    "<0.001"
4 am       1 813. 1186. 126. 19.6    " 0.002"
5 vs       1 482. 855. 118. 11.6    " 0.008"
6 gear     2 620. 993. 120. 7.47    " 0.012"
7 qsec     1 250. 623. 110. 6.03    " 0.036"
8 mpg      1 219. 592. 109. 5.28    " 0.047"
9 drat     1 75.4 448. 102. 1.82    " 0.210"
10 <none>    NA    NA    373. 99.6 NA      "  NA"

Coeficientes para los predictores del modelo obtenido:
(Intercept)    mpg      cyl6      cyl8
 -70.945    -1.489   -39.626    16.251
 disp      qsec      vs1      am1
 0.362    5.999    41.012    50.259
 gear4    gear5    carb2    carb3
-50.073   -13.168    15.190    53.043
 carb4    carb6    carb8
 95.563    132.291    178.281
```

Figura 14.8: resultado del script 14.7 con el ejemplo de regresión paso a paso.

Para el método de selección hacia adelante, en el *pipeline* de las líneas 17–18, solicitamos a la función `add1()` que evalúe todos los modelos con un solo predictor usando las variables incluidas en el modelo completo, que luego convertimos en una tabla *tibble*. Para ayudar en el análisis, damos formato a los valores p (convirtiéndolos a strings) y organizamos las filas en orden creciente de AIC. El resultado del *pipeline*, desplegada en pantalla en la línea 19, puede verse al inicio de la columna izquierda de figura 14.8. Aquí debemos decidir con cuál de los modelos nos quedaremos para continuar el proceso de construcción. Usando como criterio de selección la variable que produzca la mayor reducción de AIC, seleccionamos la variable `cyl`, que hace un aporte significativo al ajuste del modelo ($F(2, 22) = 19.6$; $p < 0.001$). La línea 22 del script actualiza el modelo agregando esta variable como su primer predictor. En la segunda iteración (líneas 25–27) encontramos que la variable `carb` es la que consigue la mayor disminución en AIC y ayuda a reducir significativamente la varianza no explicada en las observaciones ($F(5, 17) = 16.9$; $p < 0.001$), como podemos ver en la parte central de la columna izquierda de la figura 14.8, por lo que es agregada al modelo (línea 30).

Al desplegar los coeficientes de este modelo, en la línea 34 del script, vemos que se han ajustado coeficientes para siete variables indicadores (en la zona inferior de la columna izquierda de la figura 14.8), pues ambos predictores escogidos con el criterio mencionado fueron factores con 2 y 5 niveles, respectivamente.

Para la eliminación hacia atrás comenzamos con el modelo completo, e iremos quitando los predictores cuya eliminación genere el menor cambio en la varianza capturada por el modelo (que conlleva al menor valor del estadístico F). En la primera iteración (*pipeline* de las líneas 39–42), usamos la función `drop1()` para evaluar todos los modelos que resultan de la eliminación de uno de los predictores del modelo completo, encontrando

que la eliminación más prometedora corresponde a la variable `wt` ($F(1, 8) = 0,11$; $p = 0,747$), por lo que es quitada del modelo en la línea 45 del script. En la segunda iteración (líneas 48–51) encontramos que la variable `drat` puede ser eliminada del modelo (línea 54) sin afectar significativamente su ajuste ($F(1, 9) = 1,82$; $p = 0,210$). Las líneas 57–58 muestra los coeficientes resultantes para el resto de los predictores.

Si bien el script 14.7 muestra dos iteraciones de selección hacia adelante y otras dos de eliminación hacia atrás, en ambos casos podríamos seguir aplicando iteraciones hasta descartar que exista un paso que pueda mejorar el ajuste del modelo.

R cuenta con funciones que implementan los métodos para seleccionar predictores paso a paso, de manera mecánica, mediante la función `step(object, scope, direction, trace)`, que usa las funciones `add1()` y `drop1()` de manera iterativa, donde:

- **object**: modelo de regresión que es usado como punto de partida.
- **scope**: define el rango de búsqueda de los modelos. Esto puede ser una sola fórmula o una lista que contenga los componentes `lower` y `upper`, ambas fórmulas (o modelos que contengan estas fórmulas). La parte derecha de la fórmula `lower` siempre estará incluida en el modelo, mientras que la parte derecha del modelo siempre estará incluida en la de `upper`. Si solo se especifica una fórmula, esta se asigna al componente `upper` y se asume que el límite inferior es el modelo nulo. Si no se especifica `scope`, el modelo inicial se usa como el modelo `upper`.
- **direction**: indica el tipo de selección a realizar, donde `"forward"` corresponde a selección hacia adelante; `"backward"`, a eliminación hacia atrás, y `"both"`, a regresión escalonada.
- **trace**: argumento opcional que indica si se quiere ver por consola el proceso realizado.

La función `step()` utiliza el criterio de información de Akaike para guiar la búsqueda, seleccionando el predictor que conlleva al modelo con menor AIC para agregarlo al modelo cuando realiza un paso hacia adelante, o el predictor que quitándolo de él cuando realiza un paso hacia atrás. Si uno quisiera usar el criterio bayesiano de Schwarz para guiar la búsqueda, que tiende a seleccionar modelos **más simples** pero un poco **menos precisos** que con el criterio AIC, se puede lograr cambiando el valor de la penalización a la complejidad del modelo (esto es, el número de predictores considerados). Por omisión se usa el valor $k = 2$, que corresponde al usado por el AIC, que debe ser cambiado por el valor $k = \log n$, donde n corresponde al tamaño de la muestra de datos, para usar el BIC. La función `step()` acepta un argumento `k` para estos fines. Sin embargo, cualquiera sea el valor de k que especifiquemos, la función de todas formas lo reporta bajo el título **AIC**.

El script 14.8 muestra el funcionamiento de la función `step()` usando el BIC como criterio para seleccionar los predictores a incorporar en un modelo donde la respuesta es, una vez más, la potencia del motor de un automóvil construido en los años 70 que pesa entre 2 mil y 5 mil libras.

Script 14.8: regresión paso a paso (escalonada) para seleccionar los predictores a incluir en una RLM para predecir la potencia del motor en automóviles que pesan entre 2 y 5 mil libras construidos en los años 70.

```

1 library(dplyr)
2
3 # Cargar y filtrar los datos
4 datos <- mtcars |> filter(wt > 2 & wt < 5) |>
5   mutate(across(c("cyl", "vs", "am", "gear", "carb"), as.factor))
6
7 # Ajustar el modelo nulo y el modelo completo
8 nulo <- lm(hp ~ 1, data = datos)
9 completo <- lm(hp ~., data = datos)
10
11 # Realizar regresión escalonada usando el menor BIC como criterio (aunque se reporta como
12 # AIC), bajando (temporalmente) el número de cifras significativas y el ancho máximo de las
13 # líneas al imprimir.
14 opt <- options(digits = 2, width = 54)
15 modelo <- step(nulo, scope = list(lower = nulo, upper = completo),
16   direction = "both", k = log(nrow(datos)), test = "F", trace = 1)
17 options(digits = opt[[1]], width = opt[[2]])
18
19 # Mostrar los coeficientes del modelo conseguido
20 cat("\nModelo obtenido:\n")
21 print(modelo[["coefficients"]])

```

```

Start:  AIC=212
hp ~ 1

      Df Sum of Sq  RSS AIC F value Pr(>F)
+ cyl  2      68625 38549 193   19.58 1.3e-05 ***
+ qsec  1      61023 46151 194   30.41 1.3e-05 ***
+ mpg   1      60602 46572 195   29.93 1.5e-05 ***
+ disp  1      59314 47859 195   28.50 2.0e-05 ***
+ vs    1      44410 62763 202   16.27 0.00052 ***
+ gear  2      44037 63137 206    7.67 0.00297 **
+ wt    1      35016 72158 206   11.16 0.00284 **
+ carb  5      61903 45271 207    5.20 0.00359 **
<none>                 107174 212
+ drat  1      5585 101589 214    1.26 0.27240
+ am    1         2 107171 216    0.00 0.98208
---
Signif. codes:
0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Step:  AIC=193
hp ~ cyl

      Df Sum of Sq  RSS AIC F value Pr(>F)
+ carb  5      32105  6444 165   16.94 4.5e-06 ***
+ gear  2      18503 20046 183    9.23 0.0014 **
+ qsec  1      11620 26929 187    9.06 0.0067 **
+ drat  1      10241 28308 189    7.60 0.0118 *
+ am    1       9686 28863 189    7.05 0.0148 *
<none>                 38549 193
+ mpg   1       4222 34327 194    2.58 0.1230
+ wt    1       948 37600 196    0.53 0.4748
+ vs    1       388 38161 196    0.21 0.6489
+ disp  1         0 38549 196    0.00 0.9964
- cyl   2      68625 107174 212   19.58 1.3e-05 ***
---
Signif. codes:
0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Step:  AIC=165
hp ~ cyl + carb

      Df Sum of Sq  RSS AIC F value Pr(>F)
+ disp  1      3350  3094 149   17.32 0.00073 ***
+ mpg   1      1133  5311 163    3.41 0.08323 .
+ gear  2      1644  4800 164    2.57 0.10984
<none>                 6444 165
+ wt    1       463  5981 166    1.24 0.28199
+ vs    1       276  6168 167    0.72 0.41011
+ drat  1       161  6283 167    0.41 0.53064
+ am    1         8  6437 168    0.02 0.89263
+ qsec  1         6  6438 168    0.02 0.90112
- carb  5      32105 38549 193   16.94 4.5e-06 ***
- cyl   2      38827 45271 207   51.21 6.4e-08 ***
---
Signif. codes:
0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Step:  AIC=149
hp ~ cyl + carb + disp

      Df Sum of Sq  RSS AIC F value Pr(>F)
+ mpg   1         585  2510 147    3.49 0.08123 .
+ drat  1         511  2583 148    2.97 0.10540
<none>                 3094 149
+ am    1         274  2820 150    1.46 0.24588
+ gear  2         614  2480 150    1.73 0.21254
+ wt    1         79  3015 152    0.40 0.53902
+ qsec  1         71  3023 152    0.35 0.56263
+ vs    1         19  3075 152    0.09 0.76454
- disp  1      3350  6444 165   17.32 0.00073 ***
- cyl   2      6651  9745 172   17.20 0.00010 ***
- carb  5     35455 38549 196   36.67 3.3e-08 ***
---
Signif. codes:
0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Step:  AIC=147
hp ~ cyl + carb + disp + mpg

      Df Sum of Sq  RSS AIC F value Pr(>F)
+ drat  1      1263  1246 133   14.20 0.00208 **
+ am    1         818  1691 141    6.77 0.02088 *
+ gear  2         891  1618 143    3.58 0.05770 .
+ wt    1         439  2071 146    2.97 0.10704
<none>                 2510 147
+ qsec  1         233  2277 148    1.43 0.25149
- mpg   1         585  3094 149    3.49 0.08123 .
+ vs    1         17  2492 150    0.10 0.76087
- disp  1      2801  5311 163   16.74 0.00096 ***
- cyl   2      7119  9628 175   21.28 4.2e-05 ***
- carb  5     31804 34314 197   38.02 5.3e-08 ***
---
Signif. codes:
0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Step:  AIC=133
hp ~ cyl + carb + disp + mpg + drat

      Df Sum of Sq  RSS AIC F value Pr(>F)
<none>                 1246 133
+ am    1         143  1103 133    1.68 0.2170
+ wt    1         36  1210 136    0.39 0.5444
+ qsec  1          1  1245 136    0.01 0.9205
+ vs    1          1  1245 136    0.01 0.9269
+ gear  2         40  1206 139    0.20 0.8206
- drat  1      1263  2510 147   14.20 0.0021 **
- mpg   1      1337  2583 148   15.02 0.0017 **
- cyl   2      1722  2968 148    9.68 0.0023 **
- disp  1      3178  4424 162   35.70 3.4e-05 ***
- carb  5     22314 23560 190   50.14 1.9e-08 ***
---
Signif. codes:
0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Figura 14.9: salida producida por la función `step()` en el script 14.8.

Las líneas 8 y 9 del script ajustan los modelos nulo y completo para ser usados como los límites inferior y superior, respectivamente, de la regresión paso a paso. En la llamada a `step()` de las líneas 15–16, hacemos explícito el valor 1 para el argumento `trace` (que es el que tiene por omisión), por lo que la función muestra información de los modelos evaluados en cada paso. La figura 14.9 muestra la salida producida por esta llamada¹.

Notemos que la función comienza por calcular el BIC (aunque reportado como AIC) para el modelo nulo, para luego determinar las métricas que se obtendría para los diferentes modelos que se obtendrían agregando solo una de las variables disponibles. El predictor que se escoge es aquel que genera el modelo con menor BIC,

¹Las líneas inmediatamente anterior y posterior a esta llamada simplemente reducen y restauran al valor original, respectivamente, el número de cifras significativas y el el número máximo de caracteres en cada línea a ser usado por las funciones y métodos de R que imprimen en pantalla. El único propósito de esto es obtener una salida con un formato apropiado para ser mostrado en dos columnas como en estas figuras.

que corresponde al que encabeza la lista pues se muestran ordenados por este valor; en este paso, entonces, se escoge el número de cilindros del motor (`cyl`).

En la siguiente iteración, vemos que explora los diferentes modelos que se obtendrían agregando solo una del resto de las variables disponibles al modelo que incluye el predictor `cyl`, así como el que se obtendría si se eliminara esta variable. La lista es encabezada por un paso hacia adelante agregando el número de carburadores del automóvil (`carb`).

En las siguientes iteraciones se repite este procedimiento: explora los modelos que se obtendrían agregando una de las variables no incluidas en el modelo base, y los que obtendría eliminando uno de los predictores ya incluidos en él; ordena los modelos por su valor de BIC, y realiza el paso que encabeza la lista ordenada. De esta forma, agrega al modelo el volumen de los cilindros del motor (`disp`) en la iteración 3, el rendimiento del vehículo (`mpg`) en la cuarta y la relación de transmisión del eje trasero (`drat`) en la quinta.

En la iteración 6 resulta que el modelo que encabeza la lista de posibles pasos corresponde al modelo que ya se tiene, que la función `step()` marca como `<none>`, y la búsqueda se detiene. La línea 21 del script 14.8 muestra en pantalla los coeficientes del modelo que se ha conseguido, como se muestran en la figura 14.10.

Modelo obtenido:						
(Intercept)	cyl6	cyl8	carb2	carb3	carb4	carb6
-16.8129699	-35.6419700	-7.9328048	-28.8450938	14.8738567	14.4921160	90.5867539
carb8	disp	mpg	drat			
135.0690715	0.4732043	-3.6338389	38.6298876			

Figura 14.10: modelo del ejemplo obtenido con regresión escalonada.

14.4.3 Búsqueda exhaustiva

Si bien la regresión paso a paso implementada en R es bastante cómoda, la búsqueda que se realiza no es otra cosa que una implementación de una **búsqueda golosa**, por lo que no tenemos ninguna garantía de que consiga la combinación óptima de predictores, en el sentido de que permitan construir el modelo de RLM que maximiza o minimiza algún criterio.

Con el avance del poder de cómputo, se ha hecho posible una alternativa más exhaustiva que aplica un algoritmo de fuerza bruta en el que se exploran todos los subconjuntos de predictores que cumplan con algún criterio. Esto último es muy relevante, puesto que el conjunto completo de posibilidades corresponde a 2^P , siendo P la cantidad de potenciales predictores. Para nuestro ejemplo con el conjunto de datos `mtcars`, con 11 variables y 10 potenciales predictores, el conjunto a explorar contiene $2^{10} = 1.024$ combinaciones, lo que es bastante abordable en estos tiempos. Sin embargo, para un conjunto de datos con 31 o 51 columnas habría que evaluar, respectivamente, 1.073.741.824 y 1.125.899.906.842.624 combinaciones distintas, lo que es altamente costoso en tiempo de ejecución.

La función `regsubsets(formula, data, nbest, nvmax, force.in, force.out, method = "exhaustive")` del paquete `leaps` implementa este método de búsqueda exhaustiva, donde:

- **formula**: la especificación del modelo completo como se entregaría a la función `lm()`.
- **data**: matriz de datos.
- **nbest**: cantidad de modelos a reportar por cada tamaño de subconjunto. Si, por ejemplo, `nbest = 3`, entonces se reportan los tres mejores modelos con un único predictor, los tres mejores modelos con dos predictores, los tres mejores modelos con tres predictores, y así sucesivamente.
- **nvmax**: fija una cantidad máxima de predictores a considerar en el modelo.
- **force.in**: argumento opcional que toma la forma de un vector con los índices de las columnas que deben ser forzosamente consideradas en los modelos evaluados.
- **force.out**: argumento opcional que toma la forma de un vector con los índices de las columnas que deben ser forzosamente excluidas en los modelos evaluados.

Script 14.9: regresión utilizando el método de todos los subconjuntos para construir un modelo de RLM para el ejemplo con el conjunto de datos `mtcars`.

```

1 library(dplyr)
2 library(leaps)
3
4 # Cargar y filtrar los datos
5 datos <- mtcars |> filter(wt > 2 & wt < 5) |>
6   mutate(across(c("cyl", "vs", "am", "gear", "carb"), as.factor))
7
8 # Evaluar todos las combinaciones
9 combinaciones <- regsubsets(hp ~ ., data = datos, nbest = 1, nvmax = 16,
10                             method = "exhaustive")
11
12 # Graficar los resultados
13 plot(combinaciones)
14
15 # Extraer los mejores subconjuntos
16 resumen_combinaciones <- summary(combinaciones)
17 i_bic_minimo <- which.min(resumen_combinaciones[["bic"]])
18 i_r2a_maximo <- which.max(resumen_combinaciones[["adjr2"]])
19
20 mejor_comb_bic <- resumen_combinaciones[["which"]][i_bic_minimo, ]
21 mejor_comb_r2a <- resumen_combinaciones[["which"]][i_r2a_maximo, ]
22
23 # Extraer las variables seleccionadas
24 comb_mejor_bic <- names(mejor_comb_bic[mejor_comb_bic == TRUE])
25 comb_mejor_r2a <- names(mejor_comb_r2a[mejor_comb_r2a == TRUE])
26
27 # Eliminar variables indicadoras
28 nombres_mejor_bic <- unique(gsub("^(.*)\\d$", "\\1", comb_mejor_bic))
29 nombres_mejor_r2a <- unique(gsub("^(.*)\\d$", "\\1", comb_mejor_r2a))
30
31 # Obtener las fórmulas
32 pred_mejor_bic <- paste(nombres_mejor_bic[-1], collapse = " + ")
33 pred_mejor_r2a <- paste(nombres_mejor_r2a[-1], collapse = " + ")
34
35 fmla_mejor_bic <- as.formula(paste("hp", pred_mejor_bic, sep = " ~ "))
36 fmla_mejor_r2a <- as.formula(paste("hp", pred_mejor_r2a, sep = " ~ "))
37
38 # Construir y mostrar los mejores modelos
39 modelo_mejor_bic <- lm(fmla_mejor_bic, data = datos)
40 modelo_mejor_r2a <- lm(fmla_mejor_r2a, data = datos)
41
42 cat("Modelo que minimiza el BIC:\n")
43 cat("-----\n")
44 print(modelo_mejor_bic)
45 cat("\nModelo que maximiza el coeficiente de determinación ajustado:\n")
46 cat("-----\n")
47 print(modelo_mejor_r2a)

```

El script 14.9 presenta el uso del método de todos los subconjuntos al ejemplo que hemos seguido con el conjunto de datos `mtcars`. En las líneas 9–10 se llama a la función `regsubsets()` indicando que reporte solo el mejor modelo encontrado para cada tamaño del modelo, el que está permitido llegar a 16 predictores. Puede llamar la atención este último número, puesto que la matriz de datos del ejemplo solo contiene 10 posibles predictores. Esto se debe a que la búsqueda se realiza con los factores ya transformados en variables indicadoras (sin las que son redundantes).

El objeto que retorna la función `regsubsets()` contiene mucha información de los subconjuntos encontrados y es un tanto complejo darle sentido. Por esta razón, es más conveniente revisar una representación gráfica de los resultados, que puede obtenerse con la función `plot()`, como hace la línea 13 del script 14.9. La figura 14.11 muestra el gráfico generado por esta llamada. El eje x lista todas las variables disponibles para ser usadas en el modelo (de acuerdo al *scope* especificado en los argumentos `formula`, `force.in` y `force.out`, e incluyendo las variables indicadoras que se necesiten), y el eje y corresponde a una lista ordenada de los valores del BIC registrado por cada uno de los mejores modelos evaluados.

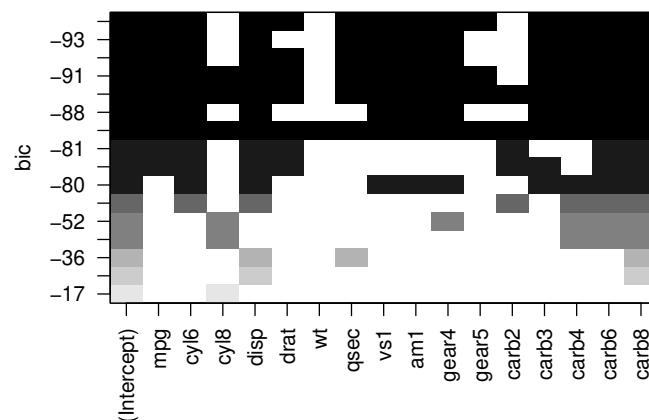


Figura 14.11: representación gráfica de las mejores combinaciones de predictores encontradas mediante el método de todos los subconjuntos para el ejemplo con el conjunto de datos `mtcars`.

Así, la figura 14.11 corresponde a una matriz en que cada fila representa a uno de los mejores subconjuntos de predictores encontrados para cada tamaño, donde se colorea el recuadro para las variables presentes en dicho modelo, con un tono más oscuro para menores valores de BIC. Notemos que el eje y se encuentra invertido, por lo que las mejores combinaciones aparecen en la parte superior de la matriz.

Específicamente para el ejemplo, como se solicitó, se muestra la mejor combinación con un predictor, la mejor con dos predictores, la mejor con tres, y así sucesivamente hasta posiblemente el modelo completo. La mejor combinación, al tope de la matriz, considera, además del intercepto, el rendimiento del vehículo (`mpg`), la variable indicadora de motor con 6 cilindros (`cyl6`), el volumen de los cilindros (`disp`), la relación de transmisión del eje trasero (`drat`), el tiempo necesitado para recorrer un cuarto de milla desde el reposo (`qsec`), el tipo de motor (`vs1`), el tipo de transmisión (`am1`), el número de marchas hacia adelante (`gear4` y `gear5`), y cuatro variables indicadoras del número de carburadores (`carb3`, `carb4`, `carb6`, y `carb8`).

Si bien el conjunto de predictores seleccionados no incluye todos niveles de algunos predictores categóricos, es preferible reconstruir el modelo con los predictores categóricos completos. Así el modelo resultante es más generalizable a datos nuevos y más fácil de interpretar por quien lo use, aún cuando sea un poco menos parsimonioso que el encontrado por la función `regsubsets()`.

En general, la tarea de reconocer las variables categóricas originales para esta reconstrucción es una **tarea manual**, pero en este caso, y solo porque todos los factores tenían niveles representados con números enteros (en vez de etiquetas textuales), se pudo hacer de manera automática con las sentencias de las líneas 24–40 del script 14.9. El resultado, puesto en pantalla por las líneas 42–47, se muestra en la figura 14.12. Podemos ver que, usando tanto el BIC o como el coeficiente de determinación ajustado como criterio para seleccionar el conjunto de predictores, se llega al mismo modelo.

14.5 CONFIABILIDAD DE UN MODELO DE RLM

Al igual que en el caso de la RLS, en la RLM las relaciones entre la variable de salida y los predictores requieren verificar condiciones importantes, alguna de las cuales son difíciles de evaluar. Afortunadamente, su cumplimiento puede transferirse a **condiciones que debe cumplir el modelo** que son más simples de comprobar. Haciendo explícitas otras consideraciones importantes para obtener un modelo de RLM confiable, podemos considerar:

1. La variable de respuesta debe ser cuantitativa y continua, sin restricciones para su variabilidad.
2. Los predictores deben ser cuantitativos o dicotómicos (de ahí la necesidad de variables indicadoras para manejar más de dos niveles).
3. Los predictores deben tener algún grado de variabilidad (su varianza no debe ser igual a cero). En otras palabras, no pueden ser constantes.

```

Modelo que minimiza el BIC:
-----

Call:
lm(formula = fmla_mejor_bic, data = datos)

Coefficients:
(Intercept)      mpg      cyl6      cyl8      disp      drat
   -109.161    -1.918   -33.626    18.317     0.385    14.756
      qsec      vs1      am1      gear4      gear5      carb2
     5.815     36.075    45.616   -47.469   -18.402     9.595
    carb3    carb4    carb6    carb8
    47.894    80.643   128.986   174.382

Modelo que maximiza el coeficiente de determinación ajustado:
-----

Call:
lm(formula = fmla_mejor_r2a, data = datos)

Coefficients:
(Intercept)      mpg      cyl6      cyl8      disp      drat
   -109.161    -1.918   -33.626    18.317     0.385    14.756
      qsec      vs1      am1      gear4      gear5      carb2
     5.815     36.075    45.616   -47.469   -18.402     9.595
    carb3    carb4    carb6    carb8
    47.894    80.643   128.986   174.382

```

Figura 14.12: modelos que resultan de seleccionar predictores con el método de todos los subconjuntos para el ejemplo.

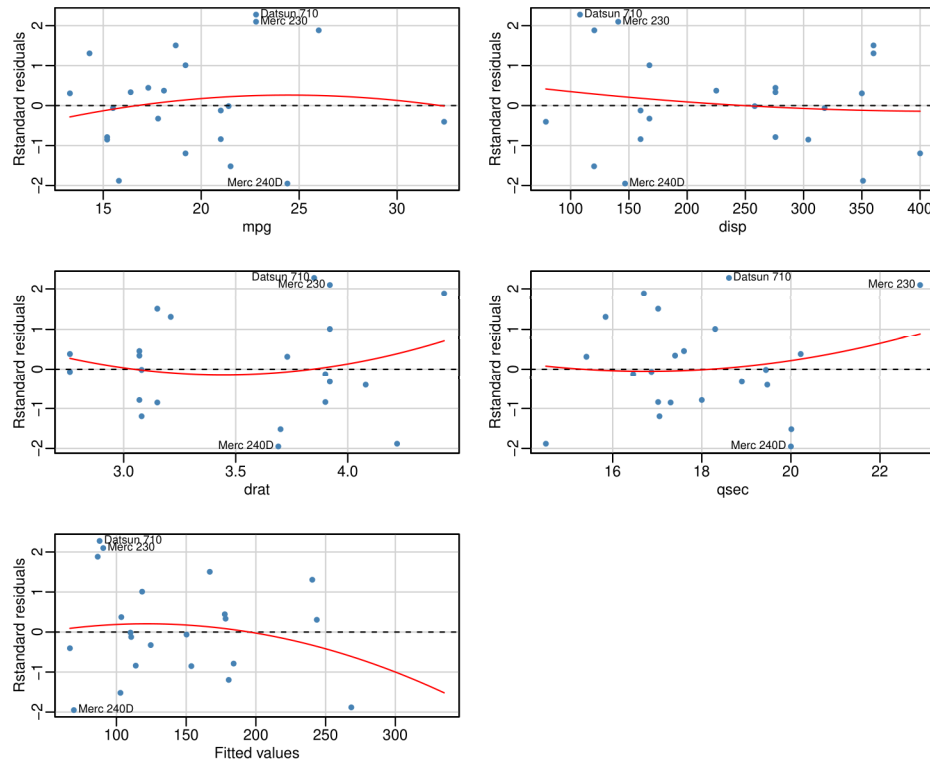
4. Cada predictor debe estar relacionado linealmente con la respuesta.
5. La distribución de los residuos debe ser cercana a la normal centrada en cero.
6. La variabilidad de los residuos debe ser aproximadamente constante (homocedasticidad).
7. Los residuos deben ser independientes entre sí.
8. No debe existir **multicolinealidad**. Esto significa que no deben darse relaciones lineales *fuertes* (coeficientes de correlación altos) entre dos o más predictores.
9. Las estimaciones de los coeficientes del modelo no debe estar alterados por unas pocas observaciones influyentes.

En general, es sencillo comprobar las primeras tres condiciones, aunque a veces puede haber duda con variables no físicas, como puntuaciones que varían dentro de un intervalo dado. También puede haber alguna dificultad con decidir si un predictor tiene al menos una escala de intervalos iguales. Además, como vimos, podemos convertir factores no dicotómicos en variables indicadoras sin mucha dificultad, al igual que comprobar que una columna de una matriz de datos no tiene un valor constante.

Para verificar las condiciones 4–6, podemos aplicar el mismo procedimiento y herramientas que usamos para evaluar modelos de RLS. Así, la figura 14.13 presenta gráficos de residuos (arriba) y gráficos marginales (abajo) para el modelo obtenido con regresión exhaustiva. El primero se obtiene con la función `residualPlots()`, mientras que la segunda con la función `marginalModelPlots()`, ambas del paquete `car`. Los parámetros gráficos utilizados se explican en 13.5.2. El único cambio requerido es dar el valor `formula("mpg + disp + drat + qsec")` al argumento `terms` en ambas llamadas, para que solo se consideren los predictores numéricos en los gráficos, ya que estos no tienen sentido para las variables categóricas.

Analizando los gráficos de residuos para cada predictor, podemos ver que aunque no muestran una clara distribución normal en torno al valor cero y hay presencia de valores atípicos, no se observan patrones sistemáticos que pudieran ser problemáticos. Podría quedar alguna duda con las curvaturas que se observan con las variables `drat` y `qsec`, además de una disminución en la dispersión de esta última, pero estas podrían ser solamente efectos visuales debidos a que no hay muchas observaciones para los valores altos de estos predictores.

Gráficos de residuos



Gráficos marginales

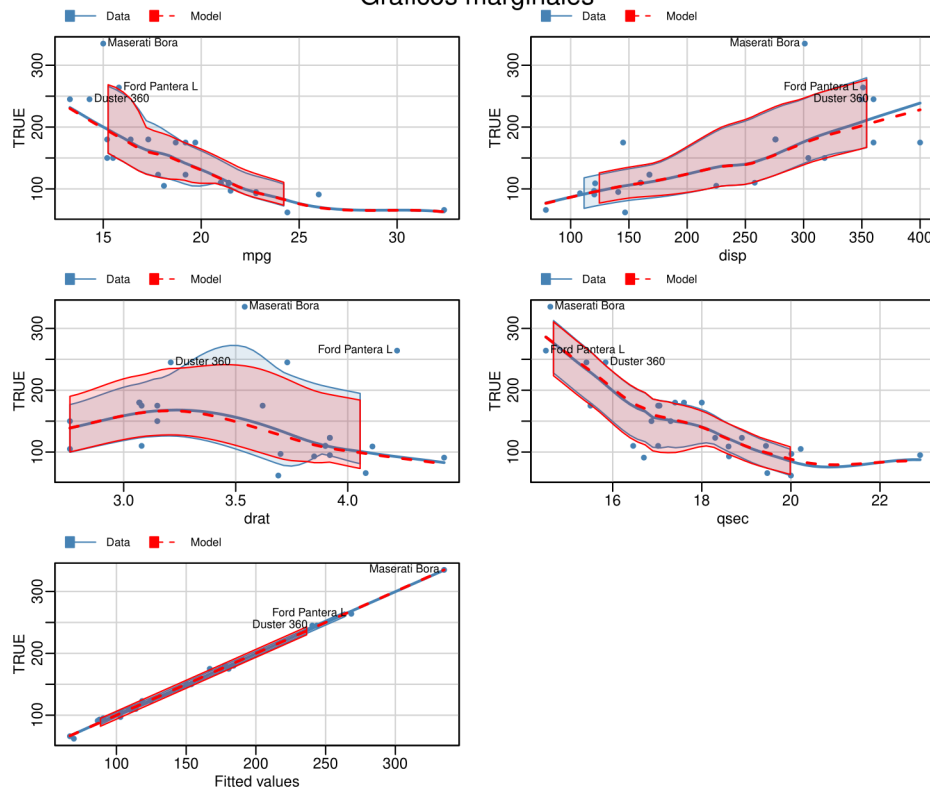


Figura 14.13: gráficos de residuos y marginales para el modelo de RLM de la figura 14.12.

Revisando los residuos del modelo ajustado, es decir para los valores predichos por el modelo (*fitted values*), vemos que tampoco muestran una distribución claramente normal, y si bien hay valores atípicos, estos no están muy lejos de las 2 desviaciones estándar, parecen mantener una dispersión constante y no muestran patrones sistemáticos. Sin embargo, aquí aparece una curvatura más notoria hacia los valores altos de potencia, aunque esta también puede ser explicada por la falta de datos (solo una observación sobre los 250 hp).

Analicemos ahora los gráficos marginales que aparecen en la parte inferior de la figura 14.13. Debemos recordar que estos muestran en colores distintos curvas que aproximan los valores observados de la variable de salida (azul) y los valores predichos por el modelo (rojo), además de comparar bandas que representan la dispersión de estos valores.

A primera vista los gráficos son preocupantes. Para el predictor `mpg` observamos una leve curvatura y una reducción de la dispersión a medida que crece el valor de la variable (rendimiento). Algo similar se observa para el predictor `qsec`. El predictor `disp` muestra una relación más lineal, pero su dispersión parece aumentar para volúmenes altos y las curvas parecen separarse en los valores altos. El gráfico para el predictor `drat` es incluso más inquietante, ya que claramente describe una curva y, si bien la dispersión parece relativamente constante, en el tramo central (aproximadamente entre 3.2 y 3.7), la banda azul se eleva formando una “joroba” que sale de la banda sombreada en rojo, lo que podría significar que el modelo está subestimando la variable de respuesta en ese rango de valores.

Sin embargo, como se mencionó anteriormente, estas curvaturas podrían deberse a la escasez de datos. Es importante entender que la línea azul se ajusta considerando solo las observaciones cercanas (dentro del vecindario), lo que le permite capturar variaciones y **curvaturas locales** de la muestra, que un modelo rígido (como una recta) pasa por alto. Por ello, en zonas con abundancia de datos la línea azul es estable y refleja la tendencia global, mientras que en zonas con observaciones muy dispersas es más fácil que la curva se distorsione o quiebre con facilidad, especialmente si entre ellas existe algún valor atípico. De hecho, en las regiones donde se concentran los datos ($15 \leq \text{mpg} \leq 25$; $75 \leq \text{disp} \leq 360$; $3 \leq \text{drat} \leq 4$; $15 \leq \text{qsec} \leq 20$), vemos que el comportamiento es aproximadamente lineal. Además, la mencionada joroba parece ser efecto de un valor atípico (Maserati Bora) que “tira” la banda azul hacia.

Es más, si nos fijamos en el gráfico marginal para los valores predichos, al estar ambos ejes en la misma escala de la variable de salida, hace patente el buen ajuste que alcanza el modelo, pues los errores son tan pequeños que parecen estar sobre la línea de regresión. Luego, el modelo puede estar fallando en el **ajuste local** (especialmente con la variable `drat`) pero parece captar bien la **tendencia global**.

Como vimos en el capítulo de RLS, podemos aplicar **pruebas de homocedasticidad** a los residuos generados por cada predictor y el modelo por medio de la función `ncvTest()`, también del paquete `car`, para descartar las dudas que tenemos con el modelo conseguido con el método de todos los subconjuntos:

```
Non-constant Variance Score Test
Variance formula: ~ mpg
Chisquare = 0.3489181, Df = 1, p = 0.55473

Non-constant Variance Score Test
Variance formula: ~ disp
Chisquare = 0.1457182, Df = 1, p = 0.70266

Non-constant Variance Score Test
Variance formula: ~ drat
Chisquare = 0.005625423, Df = 1, p = 0.94021

Non-constant Variance Score Test
Variance formula: ~ qsec
Chisquare = 0.4634622, Df = 1, p = 0.49601

Non-constant Variance Score Test
Variance formula: ~ fitted.values
Chisquare = 0.3872895, Df = 1, p = 0.53373
```

Similarmente, recordemos que la función `residualPlots()`, además de generar los gráficos de residuos, reporta pruebas que evalúan el **supuesto de linealidad**: de término cuadrático adicional para cada predictor y de no aditividad de Tukey para el modelo, las que calculan un estadístico *t* de Student y el correspondiente valor

p para la contribución que haría en el ajuste la inclusión de los **valores al cuadrado** del predictor o de los valores predichos, respectivamente. Para el modelo del ejemplo se obtiene:

	Test stat	Pr(> Test stat)
mpg	-0.0657	0.9492
disp	0.0266	0.9795
drat	1.1098	0.2993
qsec	0.7801	0.4578
Tukey test	-0.8035	0.4217

De esta forma, no podemos descartar que las condiciones 4, 5 y 6 (relaciones lineales, normalidad y homocedasticidad de los residuos) se estén cumpliendo para el modelo del ejemplo (aunque la comprobación de la condición 5 fue solamente visual y podría formalizarse con pruebas de hipótesis).

Pasando a la condición 7, nuevamente podemos utilizar lo hecho para RLS y aplicar la prueba de Durbin-Watson implementada en la función `durbinWatsonTest()` del paquete `car`, obteniendo:

lag	Autocorrelation	D-W Statistic	p-value
1	-0.2912021	2.581483	0.752

Alternative hypothesis: rho != 0

Con esto descartamos que exista autocorrelación entre los residuos, por lo que no hay evidencia de que no se esté cumpliendo la condición de independencia.

La condición 8 no es realmente una imposición de la RLM, pero evitar la multicolinealidad es importante porque el método para ajustar los coeficientes de un modelo asume que puede cambiar el valor para una variable predictora, **manteniendo los otros constantes**. Cuando las variables predictoras están correlacionadas, se hace imposible cambiar el valor de un coeficiente sin alterar también los de las demás variables, desestabilizando la estimación de cómo influye cada una en la variable de salida. Así, cuando existe multicolinealidad, los valores de los coeficientes varían enormemente si se agregan o quitan unos pocos datos de entrenamiento o, peor aún, se cambia el orden en que se agregan los predictores al modelo. En consecuencia, la multicolinealidad afecta la calidad del modelo al reducir su poder estadístico y su generalidad.

Por esta razón, necesitamos contar con alguna estrategia que nos permita escoger un conjunto de predictores lo más independientes entre sí para que puedan contribuir con información no correlacionada al modelo de RLM. La alternativa más usada se basa en el **factor de inflación de varianza** (VIF, por *variance inflation factor*) y el estadístico **tolerancia** ($1/\text{VIF}$).

El VIF para el i -ésimo predictor se calcula mediante la ecuación 14.6, donde R_i^2 corresponde al coeficiente de determinación de una regresión auxiliar que se obtiene usando todos los predictores restantes para ajustar una RLM con el predictor i como respuesta (Fox & Weisberg, 2018).

$$\text{VIF}_i = \frac{1}{1 - R_i^2} \quad (14.6)$$

Sin embargo, la definición de la ecuación 14.6 no es adecuada para modelos de regresión que incluyen predictores categóricos codificados con múltiples variables indicadoras. Para estos modelos se debe calcular **factores de inflación de varianza generalizados** (GVIF) utilizando la ecuación 14.7, donde $\mathbf{C}_{\text{full}}$ corresponde a la matriz de covarianza del modelo completo y $\mathbf{C}_{-j}$ a la matriz de covarianza del modelo sin el predictor j (Fox & Weisberg, 2018).

$$\text{GVIF}_i = \frac{\det(\mathbf{C}_{\text{full}})}{\mathbf{C}_{-j}} \quad (14.7)$$

Además, los GVIF pueden ajustarse de forma que queden en la misma escala que los factores de inflación de la varianza usuales utilizando la relación $\text{GVIF}_{\text{adj.}} = \text{GVIF}^{1/(2 \, df)}$, donde df son los grados de libertad del predictor, igual al número de coeficientes que aporta al modelo (1 para variables continuas y $d - 1$ para variables categóricas con d niveles). Cuando se usan $\text{GVIF}_{\text{adj.}}$, estos valores reemplazan a los valores normales de VIF al calcular la tolerancia de cada predictor.

Aunque no hay un acuerdo general, es común encontrar los siguientes criterios para interpretar los valores de VIF (o $GVIF_{adj}$):

- $VIF = 1$: no hay multicolinealidad.
- $1 < VIF \leq 5$: existe multicolinealidad moderada que podría afectar ligeramente los resultados, pero generalmente no es motivo de gran preocupación.
- $5 < VIF \leq 10$: la multicolinealidad es preocupante y podría afectar significativamente los resultados, por lo que sería recomendable investigar más a fondo y considerar acciones correctivas.
- $VIF > 10$: la multicolinealidad es severa y debe abordarse pues es probable que los coeficientes sean inestables y las inferencias estadísticas poco fiables.

En el caso de la tolerancia, la literatura sugiere que valores bajo 0,2 podrían ser problemáticos, pues esto puede interpretarse como que el 80 % de la varianza de ese predictor está explicada por los demás predictores del modelo, una clara señal de multicolinealidad. Valores cercanos a 0,4 también deberían ser revisados.

El paquete `car` de R incluye la función `vif(mod)` para calcular los factores de inflación de la varianza (o los generalizados y ajustados, según corresponda) para un modelo de RLM. Al usarla para el modelo ejemplo (figura 14.12), obtenemos:

	GVIF	Df	$GVIF \sim (1/(2*Df))$
mpg	7.446394	1	2.728808
cyl	135.733676	2	3.413280
disp	23.904251	1	4.889197
drat	15.759580	1	3.969834
qsec	12.433984	1	3.526185
vs	16.194473	1	4.024236
am	14.742095	1	3.839544
gear	115.447392	2	3.277902
carb	854.968528	5	1.964242

Podría ser prudente revisar si un modelo que no considere la variable `disp`, que es la que muestra un valor cercano a 5, podría resultar más conveniente. Tarea que dejaremos pendiente para los ejercicios del capítulo.

La última condición para contar con un modelo de RLM confiable en realidad tampoco es una restricción de la RLM pero, como vimos para la RLS, la presencia de observaciones sobreinfluyentes afectan las estimaciones del modelo y su generalización a datos no vistos.

Siguiendo lo realizado con modelos de RLS, podemos utilizar la función `influencePlot()` del paquete `car`, de la misma forma que hicimos en 13.5.3, para obtener una representación gráfica de posibles valores atípicos, el apalancamiento y distancia de Cook de los residuos generados por el modelo.

La figura 14.14 muestra el gráfico obtenido para el modelo ejemplo que estamos analizando. En pantalla se identifican los siguientes casos posiblemente problemáticos:

	StudRes	Hat	CookD
Datsun 710	3.293151	0.8805307	2.3858412
Merc 240D	-2.420409	0.6464100	0.4347061
Merc 230	2.766756	0.8900778	2.2271810
Ford Pantera L	-2.278866	0.8664138	1.4360599
Ferrari Dino	NaN	1.0000000	NaN
Maserati Bora	NaN	1.0000000	NaN
Volvo 142E	NaN	1.0000000	NaN

Vemos que hay tres casos con apalancamiento igual a 1! (y ¡distancia de Cook indefinida!) y otros tres con distancias de Cook muy superiores al umbral (menos exigente) 1. Es muy probable que la regresión esté dominada por estos casos. Sin embargo, el problema podría ser que el modelo está **sobreajustado** (*overfitting* en inglés), por lo que hace predicciones perfectas (cero error) o casi perfectas con estos datos, pero probablemente sea incapaz de generalizar a datos no visto.

Otra posibilidad podría ser que existe una multicolinealidad severa entre los predictores del modelo, que la métrica $GVIF_{adj}$ no logra detectar con claridad. Por supuesto que también podría tratarse simplemente de valores atípicos que están distorsionando el ajuste del modelo para la mayoría de los casos.

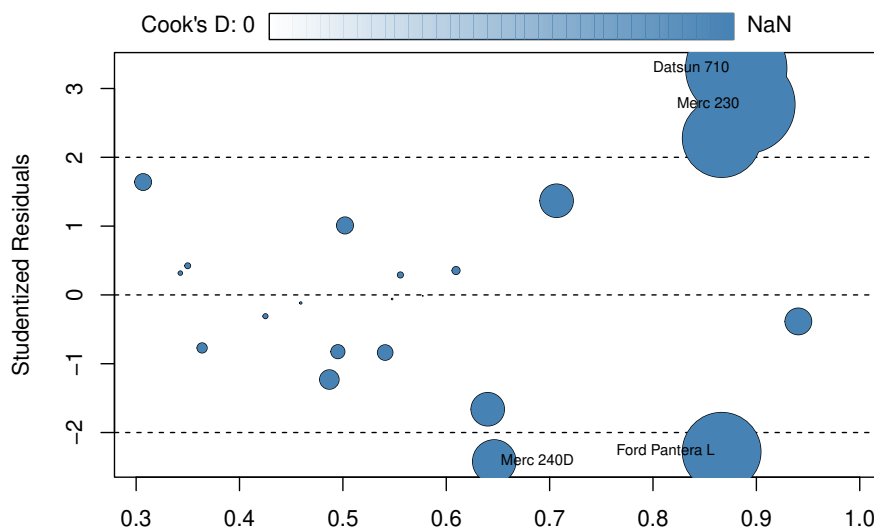


Figura 14.14: gráfico de influencia para el modelo de RLM de la figura 14.12.

Como se dijo al discutir la RLS, eliminar las observaciones problemáticas podría constituir una acción reñida con la ética profesional si la decisión no se basa en un análisis riguroso de los datos.

De hecho, para el ejemplo, se da una combinación de sobreajuste, multicolinealidad y casos excepcionales. Se requieren varias iteraciones seleccionando predictores para ser quitados y evaluando modelos para conseguir uno más confiable. Queda como desafío para el lector o lectora realizar estas iteraciones.

Aquí vale la pena hacer un comentario sobre otro aspecto importante a tener en cuenta al momento de determinar si un modelo de RLM es confiable: el **tamaño de la muestra: no existe** una fórmula matemática o un teorema que permita determinar el tamaño de la muestra que necesitamos para construir un modelo de regresión confiable y útil, del principio general de que un mayor número de observaciones mejora la precisión. Sin embargo, en la práctica suele utilizarse como guía orientativa la recomendación de contar con al menos 10 o 15 observaciones por cada predictor numérico y por cada nivel de las variables categóricas. De acuerdo a este criterio, la muestra del ejemplo, que cuenta con 25 observaciones, está muy lejos de ser suficiente para que podamos conseguir un buen modelo de 15 predictores (considerando las variables indicadores), como el del ejemplo.

14.6 CALIDAD PREDICTIVA DE UN MODELO DE RLM

Al igual que en el caso de regresión lineal simple, podemos usar la validación cruzada como herramienta para mejorar la estimación del error cuadrado medio que consigue un modelo de RLM. No hay diferencia en la realización de este proceso con respecto a lo estudiado para la RLS, por lo que no se aborda aquí con mayor detalle.

Como ejercicio académico, y sabiendo que tiene problemas, el script 14.10 ajusta el modelo del ejemplo obtenido con búsqueda exhaustiva en la sección anterior, usando validación cruzada dejando uno fuera (líneas 13–15). En la línea 16 se obtiene el modelo final, mostrando en pantalla la fórmula (línea 21), el resultado en cada pliegue (línea 24) y el error global estimado para el modelo (línea 27). La figura 14.15, muestra el resultado que se obtiene. Notemos los mensajes de advertencia (*warnings*) que salen a pantalla al realizar esta operación, pues la función `predict()`, que internamente llama la función `train()`, detecta que el modelo que se está utilizando tiene problemas de ajuste.

Para comparar, el script 14.11 continúa el script anterior repitiendo el proceso para un modelo con un predictor menos, específicamente quitando el factor `carb`. El resultado se muestra en la figura 14.16. Vemos que pudimos conseguir que ahora no aparezcan advertencias de que el modelo tiene problemas y que este parece conseguir una mejor calidad predictiva (menor RMSE).

Script 14.10: validación cruzada dejando uno fuera para construir el modelo de RLM de la figura 14.12.

```

1 library(caret)
2 library(dplyr)
3
4 # Imprimir mensajes de advertencia a medida que ocurren
5 opt <- options(warn = 1)
6
7 # Cargar y filtrar los datos
8 datos <- mtcars |> filter(wt > 2 & wt < 5) |>
9   mutate(across(c("cyl", "vs", "am", "gear", "carb"), as.factor))
10
11 # Ajustar y mostrar el modelo usando validación cruzada dejando uno fuera
12 set.seed(111)
13 fmla <- formula("hp ~ mpg + cyl + disp + drat + qsec + vs + am + gear + carb")
14 entrenamiento <- train(fmla, data = datos, method = "lm",
15   trControl = trainControl(method = "LOOCV"))
16 modelo <- entrenamiento[["finalModel"]]
17
18 # Mostrar la fórmula y las predicciones del modelo
19 cat("Modelo obtenido con regsubset():\n")
20 cat("-----\n\n")
21 print(fmla)
22
23 cat("\nPredicciones en cada pliegue:\n")
24 print(entrenamiento[["pred"]])
25
26 cat("\nError estimado para el modelo:\n")
27 print(entrenamiento[["results"]])

```

```

Warning in predict.lm(modelFit, newdata) :
  prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases
Warning in predict.lm(modelFit, newdata) :
  prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases
Warning in predict.lm(modelFit, newdata) :
  prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases
Modelo obtenido con regsubset():
-----

```

```
hp ~ mpg + cyl + disp + drat + qsec + vs + am + gear + carb
```

Predicciones en cada pliegue:

	pred	obs	rowIndex	intercept
Mazda RX4	111.08386	110	1	TRUE
Mazda RX4 Wag	117.61153	110	2	TRUE
Datsun 710	50.61264	93	3	TRUE
Hornet 4 Drive	110.14338	110	4	TRUE
Hornet Sportabout	163.36360	175	5	TRUE
Valiant	101.15911	105	6	TRUE
Duster 360	229.46875	245	7	TRUE
Merc 240D	83.11698	62	8	TRUE
Merc 230	54.26651	95	9	TRUE
Merc 280	113.79861	123	10	TRUE
Merc 280C	125.77044	123	11	TRUE
Merc 450SE	177.34866	180	12	TRUE
Merc 450SL	176.45313	180	13	TRUE
Merc 450SLC	186.36359	180	14	TRUE
Fiat 128	76.69676	66	15	TRUE
Toyota Corona	113.30605	97	16	TRUE
Dodge Challenger	150.60757	150	17	TRUE
AMC Javelin	158.09354	150	18	TRUE
Camaro Z28	242.04393	245	19	TRUE
Pontiac Firebird	185.74943	175	20	TRUE
Porsche 914-2	57.84787	91	21	TRUE
Ford Pantera L	297.15213	264	22	TRUE
Ferrari Dino	46.01351	175	23	TRUE
Maserati Bora	160.61755	335	24	TRUE
Volvo 142E	-65.38245	109	25	TRUE

Error estimado para el modelo:

	intercept	RMSE	Rsquared	MAE
1	TRUE	58.21662	0.4884008	30.48335

Figura 14.15: resultado de aplicar validación cruzada dejando uno fuera para construir el modelo de RLM de la figura 14.12.

Script 14.11: (continuación del script 14.10) validación cruzada dejando uno fuera para construir el modelo de RLM de la figura 14.15 con un predictor menos (`carb`).

```

29 # Ajustar y mostrar el modelo usando validación cruzada
30 # dejando uno fuera sin la variable "carb".
31 set.seed(111)
32 fmla <- formula("hp ~ mpg + cyl + disp + drat + qsec + vs + am + gear")
33 entrenamiento <- train(fmla, data = datos, method = "lm",
34                       trControl = trainControl(method = "LOOCV"))
35 modelo <- entrenamiento[["finalModel"]]
36
37 # Mostrar la fórmula y las predicciones del modelo modificado
38 cat("\n\nModelo con un predictor menos:\n")
39 cat("-----\n\n")
40 print(fmla)
41 cat("\n")
42
43 cat("Predicciones en cada pliegue:\n")
44 print(entrenamiento[["pred"]])
45
46 # Mostrar el resultado estimado para el modelo
47 cat("\nError estimado para el modelo:\n")
48 print(entrenamiento[["results"]])
49
50 # Reestabler opción para warnings
51 options(warn = opt[[1]])

```

```

Modelo con un predictor menos:
-----

hp ~ mpg + cyl + disp + drat + qsec + vs + am + gear

Predicciones en cada pliegue:
      pred obs rowIndex intercept
Mazda RX4      101.73743 110         1      TRUE
Mazda RX4 Wag    97.09869 110         2      TRUE
Datsun 710      117.71142  93         3      TRUE
Hornet 4 Drive  102.18601 110         4      TRUE
Hornet Sportabout 176.99516 175         5      TRUE
Valiant        121.90452 105         6      TRUE
Duster 360     194.65422 245         7      TRUE
Merc 240D       78.85562  62         8      TRUE
Merc 230      -28.96070  95         9      TRUE
Merc 280       135.85810 123        10      TRUE
Merc 280C      143.56208 123        11      TRUE
Merc 450SE     178.62351 180        12      TRUE
Merc 450SL     170.90247 180        13      TRUE
Merc 450SLC    184.48076 180        14      TRUE
Fiat 128       -10.89509  66        15      TRUE
Toyota Corona   62.07997  97        16      TRUE
Dodge Challenger 197.87334 150        17      TRUE
AMC Javelin     196.45208 150        18      TRUE
Camaro Z28     203.51950 245        19      TRUE
Pontiac Firebird 177.12708 175        20      TRUE
Porsche 914-2   156.94908  91        21      TRUE
Ford Pantera L  353.64082 264        22      TRUE
Ferrari Dino    150.86278 175        23      TRUE
Maserati Bora   241.30126 335        24      TRUE
Volvo 142E     129.74939 109        25      TRUE

Error estimado para el modelo:
  intercept      RMSE  Rsquared      MAE
1      TRUE 47.14171 0.6214021 34.24198

```

Figura 14.16: resultado de aplicar validación cruzada dejando uno fuera para construir el modelo de RLM de la figura 14.15 sin el predictor `carb`.

14.7 EJERCICIOS PROPUESTOS

- 14.1 Revise si eliminando algunos predictores del modelo de RLM de la figura 14.12, y tal vez algunos casos influyentes, es posible conseguir un modelo de RLM confiable para el conjunto de datos `mtcars` del ejemplo.
- 14.2 Tome una muestra de 100 observaciones del conjunto de datos `Prestige` del paquete `carData`, y construya un modelo de RLM con dos o tres predictores para la variable de salida `income` y evalúa su confiabilidad y calidad predictiva usando validación cruzada simple.
- 14.3 Tome una muestra de 100 observaciones del conjunto de datos `Prestige` del paquete `carData`, y construya un modelo de RLM con dos o tres predictores para la variable de salida `prestige` y evalúa su confiabilidad y calidad predictiva usando validación cruzada simple de 4 pliegues.
- 14.4 Tome una muestra de 100 observaciones del conjunto de datos `Prestige` del paquete `carData`, y construya un modelo de RLM con dos o tres predictores para la variable de salida `education` y evalúa su confiabilidad y calidad predictiva usando validación cruzada dejando uno fuera.
- 14.5 Tome una muestra de 250 observaciones del conjunto de datos `Boston` del paquete `MASS`, y construya un modelo de RLM con dos o tres predictores para la variable de salida `medv` y evalúa su confiabilidad y calidad predictiva usando validación cruzada simple.
- 14.6 Tome una muestra de 250 observaciones del conjunto de datos `Boston` del paquete `MASS`, y construya un modelo de RLM con dos o tres predictores para la variable de salida `rm` y evalúa su confiabilidad y calidad predictiva usando validación cruzada de 10 pliegues.
- 14.7 Tome una muestra de 250 observaciones del conjunto de datos `Boston` del paquete `MASS` que tengan valores menores a 400 en la variable de salida `tax`, y construya un modelo de RLM con dos o tres predictores para predecirla. Luego evalúa su confiabilidad y calidad predictiva usando validación cruzada dejando uno fuera.